

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie

Wydział Informatyki

Praca dyplomowa

System wspomagania pracy inżynierów gospodarki wodnej

A system for supporting the work of water management engineers

Autor: **Adam Kamil Ćwikła**
Kierunek: **Informatyka - Data Science**
Opiekun pracy: **Włodzimierz Funika, dr inż.**

Kraków, 2025

W trakcie przygotowywania niniejszej pracy magisterskiej wykorzystano narzędzia językowe wspomagane sztuczną inteligencją w celu poprawy klarowności, spójności oraz ogólnej jakości tekstu. Narzędzia te wspierały m.in. korektę gramatyczną, udoskonalanie stylu oraz sugerowały sformułowania. Jednakże treść, pomysły, wyniki badań oraz wnioski przedstawione w pracy są wyłącznie autorstwa autora. Za wszelkie błędy lub pominięcia odpowiada autor.

Chciałbym wyrazić wdzięczność mojemu promotorowi, dr. Włodzimierzowi Funice, za pomoc redakcyjną i merytoryczną przy mojej pracy. Wyrazy podziękowania kieruję również do mgr inż. Pawła Koperka, za cenne dyskusje oraz wsparcie w rozwiązywaniu problemów technicznych napotkanych podczas realizacji badań.
Pragnę również wyrazić wdzięczność moim rodzicom za nieocenione wsparcie emocjonalne podczas przygotowywania tej pracy. Szczególne podziękowania kieruję także do mojego wujka, Janusza Filipczyka, z którym wspólnie badałem skuteczność systemu.

Streszczenie

Aktualnie rośnie użycie modeli LLM w dziedzinie przetwarzania języka. Najczęściej korzysta się z nich w ramach czatbotów, które udzielają informacji na różne zagadnienia m.in. inżynierii wodnej. Niestety, nie zawsze podane przez nie informacje są poprawne. Niniejsza praca miała na celu zbadanie, jak precyzyjne mogą być systemy oparte na modelach LLM.

Zaprojektowano prototyp systemu RAG bazującego na dużym modelu językowym wraz z dostępem do bazy wektorowej zawierającej dodatkowe informacje potrzebne do odpowiedzi na pytania. Przeanalizowano, w jaki sposób zastosowanie dodatkowych modułów oraz metod poprawy jakości systemu wpływa na poprawność generowanych odpowiedzi.

Wyniki eksperymentów wykazały, że zastosowanie modułu re-rankingu znacząco zwiększa precyzję, z jaką system udziela odpowiedzi. System wzbogacony o indeksowanie z rodzicem był w stanie udzielać informacji opartych na źródłach, nie pomijając istotnych szczegółów. Zweryfikowano również wpływ samego modelu językowego na składanie odpowiedzi, stwierdzając, że gpt-4o zapewnia najlepsze rezultaty. Dodatkowo oceniono skuteczność klasycznych metryk przetwarzania języka, takich jak ROUGE, BLEU czy odległości Hamminga, Levenshteina, Jaro i Jaro-Winklera, aby sprawdzić ich przydatność w ocenie systemu.

Wybór odpowiedniego modelu językowego czy dodatkowych modułów jest kluczowy dla zwiększenia poprawności odpowiedzi. Poprzez różne metody i moduły można skutecznie dostosować się do tekstu źródłowego, zapewniając pełniejszy kontekst do generowania odpowiedzi.

Słowa kluczowe

Retrieval-augmented generation (RAG), duży model językowy (LLM), multi query, re-ranking, indeksowanie z rodzicem, indeksowanie podstawowe, model embedujący, przetwarzanie języka, fragmentowanie, istotne dokumenty, overlapping, Langchain, Langgraph, Qdrant, Ragas, Python, twierdzenie, prompt, halucynowanie, inżynieria wodna.

Abstract

Currently, the use of LLM models in the field of natural language processing is increasing. They are most often applied in chatbots that provide information on various topics, including, for example, water engineering. Unfortunately, the information they provide is not always correct. This thesis aims to examine how accurate systems based on LLMs can be.

A prototype RAG system was designed, based on a large language model with access to a vector database containing additional information needed to answer questions. An analysis was conducted to determine how the use of additional modules and methods for improving system quality affects the correctness of generated responses.

The results demonstrated that applying a re-ranking module significantly increases the precision of the system's responses. The system, enhanced with parent indexing, was able to provide information based on sources without omitting important details. The impact of the language model itself on answer generation was also verified, showing that GPT-4o delivers the best results. Additionally, the effectiveness of classical NLP metrics such as ROUGE, BLEU, and the Hamming, Levenshtein, Jaro, and Jaro–Winkler distances was evaluated to assess their usefulness in system evaluation.

The choice of an appropriate language model and additional modules is crucial for improving response accuracy. Through various methods and modules, it is possible to effectively adapt to the source text, ensuring a more comprehensive context for generating answers.

Keywords

Retrieval-augmented generation (RAG), large language model (LLM), multi query, re-ranking, parent indexing, basic indexing, embedding model, natural language processing, chunking, relevant documents, overlapping, Langchain, Langgraph, Qdrant, Ragas, Python, claim, prompt, hallucination, hydraulic engineering.

Spis treści

Spis rysunków xiii

Spis tabel xv

Lista kodów źródłowych xvii

1. Wstęp 1
- 1.1. Tło badań . . . 1
- 1.2. Cel pracy . . . 2
- 1.3. Zakres prac . . . 3

2. Przegląd literatury w zakresie Retrieval-Augmented Generation (RAG) 5
- 2.1. Przegląd publikacji . . . 5
  - 2.1.1. Pierwsze wzmianki o RAG . . . 5
  - 2.1.2. Wzmacnianie systemu . . . 7
  - 2.1.3. Ewaluacja modelu RAG . . . 9
  - 2.1.4. Optymalizacja modelu RAG . . . 11
- 2.2. Podsumowanie . . . 12

3. Metodologia badań 13
- 3.1. Badanie działania RAG . . . 13
- 3.2. Metryki LLM . . . 14
  - 3.2.1. Precyzja kontekstu . . . 14
  - 3.2.2. Recall kontekstu . . . 14
  - 3.2.3. Recall encji . . . 15
  - 3.2.4. Wrażliwość na szum . . . 15
  - 3.2.5. Wierność . . . 15
  - 3.2.6. Zgodność z faktami . . . 16
- 3.3. Metryki przetwarzania języka . . . 16
  - 3.3.1. BLEU . . . 17
  - 3.3.2. ROUGE . . . 19
- 3.4. Metryki z modelem embeddującym . . . 19
  - 3.4.1. Podobieństwo semantyczne . . . 19
  - 3.4.2. Trafność odpowiedzi . . . 19

3.5. Podsumowanie 20

4. Koncepcja architektoniczna rozwiązania **21**
4.1. Proces segmentacji tekstu 21
4.2. Osadzenie i indeksowanie podzielonych fragmentów 22
4.3. Baza danych wektorowych 26
4.3.1. Wprowadzenie 26
4.3.2. Dostępne silniki baz danych na rynku 26
4.3.3. *Qdrant* 26
4.3.4. *ChromaDB* 30
4.3.5. *Pgvector* 30
4.4. Schemat pełnego modelu RAG 30
4.4.1. Wybór modelu językowego oraz bibliotek 30
4.4.2. Wierzchołek *generate_query_context* 32
4.4.3. Wierzchołek *retrieve* 34
4.4.4. Wierzchołek *generate_user_answer* 37
4.4.5. Wierzchołek *save_answer* 39
4.4.6. Połączenie kroków w graf 39
4.5. Podsumowanie 40

5. Eksperymenty, założenia i ich cele **41**
5.1. Założenia 41
5.2. Cele 42
5.3. Przebieg eksperymentów 43
5.4. Studium przypadku 44
5.4.1. Krok 1: Otrzymanie pytania od użytkownika 44
5.4.2. Krok 2: Multi query 45
5.4.3. Krok 3: Pobranie danych z bazy wektorowej 46
5.4.4. Krok 4: Re-ranking 46
5.4.5. Krok 5: Zwrócenie odpowiedzi użytkownikowi 46
5.5. Podsumowanie 47

6. Ewaluacja **49**
6.1. Pytania badawcze 49
6.2. Wyniki ewaluacji z wykorzystaniem metryk 49
6.2.1. Precyzja 50
6.2.2. Recall 50
6.2.3. Wrażliwość na szum 51
6.2.4. Podobieństwo semantyczne 51
6.2.5. Wierność 52

6.2.6. Trafność odpowiedzi . . . 52
6.2.7. Zgodność z faktami . . . 53
6.2.8. ROUGE . . . 54
6.2.9. BLEU . . . 56
6.2.10. Metryki odległości tekstowej . . . 57
6.2.11. Porównanie sposobów indeksowania . . . 58
6.2.12. Porównanie wyników dla różnych modeli językowych . . . 59
6.3. Ewaluacja z ekspertem . . . 61
6.4. Podsumowanie . . . 63

7. Podsumowanie **65**
7.1. Osiągnięcia . . . 65
7.2. Wnioski z przeprowadzonych badań . . . 65
7.3. Przyszłe prace . . . 68

Bibliografia **73**

Spis rysunków

1.1. Liczba użytkowników *czatbota* od *OpenAI* (źródło [1]) 2

2.1. Schemat AssistantRAG (źródło [3]) 8
2.2. Podział benchmarków (źródło [4]) 10

3.1. Podział metryk wraz z przykładami. 13

4.1. Przykład overlappingu dla artykułu 201 Prawa wodnego (źródło [10]) 22
4.2. Indeksowanie podstawowe 23
4.3. Indeksowanie z multi-wektorem 24
4.4. Indeksowanie z rodzicem 25
4.5. Pojedynczy rekord bazy *Qdrant* 27
4.6. Algorytm *NSW* (źródło [15]) 28
4.7. Lista z przeskokami (źródło [15]) 29
4.8. Algorytm *HNSW* (źródło [15]) 29
4.9. Pełny schemat stworzonego systemu RAG 31
4.10. Przykład stworzonych pytań za pomocą multi query 34

5.1. Najważniejsza część myśli z pytania użytkownika 44

6.1. Ocena systemu z wykorzystaniem poszczególnych metryk 54
6.2. Wykres podsumowujący metrykę *ROUGE* 56
6.3. Porównanie wyników systemu dla poszczególnych metryk odległości 57
6.4. Wyniki systemu dla poszczególnych metod indeksowania 58
6.5. Wydajność systemu dla różnych modeli językowych 59
6.6. Porównanie wyników dla rozwiązania systemu AquaRAG dla modeli oceniających 61

Spis tabel

6.1. Porównanie systemów RAG dla **precyzji**. 50
6.2. Porównanie systemów RAG dla ***recall***. 50
6.3. Porównanie systemów RAG dla **wrażliwości na szum**. 51
6.4. Porównanie systemów RAG dla **podobieństwa semantycznego**. 52
6.5. Porównanie systemów RAG dla **wierności**. 52
6.6. Porównanie systemów RAG dla **trafności odpowiedzi**. 53
6.7. Porównanie systemów RAG dla **zgodności z faktami**. 53
6.8. Porównanie systemów pod względem ***ROUGE precyzja***. 55
6.9. Porównanie systemów pod względem ***ROUGE recall***. 55
6.10. Porównanie systemów pod względem ***ROUGE F1***. 55
6.11. Porównanie systemów pod względem ***BLEU***. 57

Lista kodów źródłowych

4.1. Stan aplikacji, w każdym z kroków 32
4.2. Przykładowa funkcja narzędzi 32
4.3. Stan aplikacji, w każdym z kroków 39
4.4. Funkcja route_tools 40

5.1. Przykładowe pytanie ze zbioru 41
5.2. Przykładowa odpowiedź systemu 43

1. Wstęp

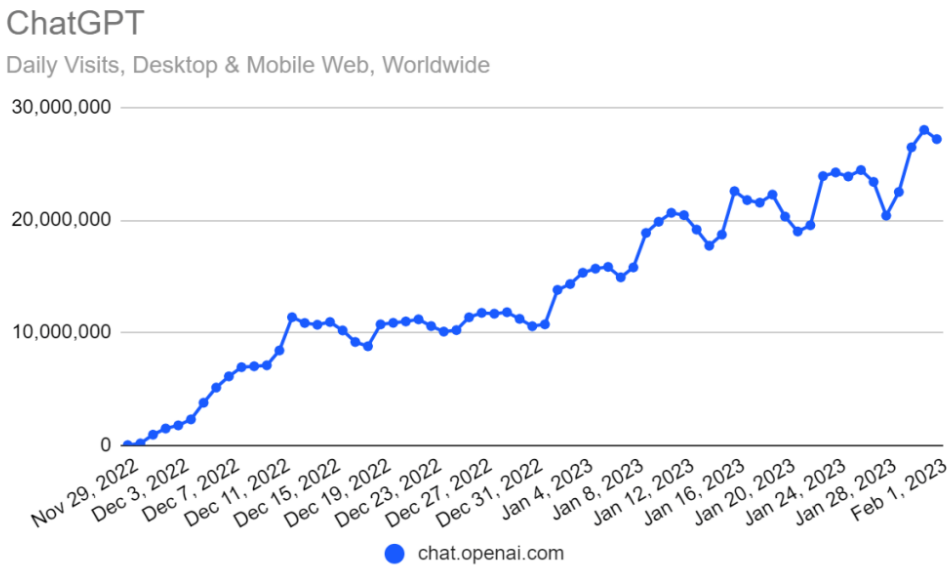
Ten rozdział stanowi wprowadzenie do omawianego tematu, prezentując główny problem, który jest przedmiotem tej pracy. Dla wskazanego problemu w kolejnych rozdziałach przeprowadzono eksperymenty. Dodatkowo określono zakres prac, które zostały poczynione, aby ten cel zrealizować.

1.1. Tło badań

W ostatnich latach duże modele językowe (LLM) zyskały ogromną popularność dzięki zdolności do przetwarzania i generowania naturalnego języka.

Głównym aspektem, gdzie takie modele znalazły swoje zastosowanie, to *czatboty*. Dzięki swojej możliwości generowania niesprawdzonych informacji na zadany przez użytkownika temat można wygenerować rozmowę między maszyną a człowiekiem. Potem, bazując na zapisie *czatu*, ciężko jest jasno określić, która strona jest robotem. Zagadnienie to poruszył już sam *Turing* w 1950 r. Stworzył tak zwany test (nazwany później jego nazwiskiem), bazujący na rozmowie z robotem, mający na celu sklasyfikowanie rozmówców. Obecnie na rynku powstało wiele takich robotów, które są badane przez ów test, są to m.in.

- *ChatGPT – OpenAI (USA)* – najpopularniejsze modele językowe na rynku. Zawierają wiele udoskonaleń związanych z *czatem* pozwalających użytkownikowi dopasowywać je do swoich potrzeb.
- *DeepSeek – DeepSeekAI (Chiny)* – model open source rozwijany w Chinach. Pomimo swojej początkowej kontrowersji zaskarbił sobie miejsce dzięki darmowemu dostępowi do modeli z wielką ilością parametrów.
- *Mistral – MistralAI (Francja)* – głównie lekkie modele, językowe o wysokiej jakości generacji. Służące do stworzenia prostych aplikacji i szybkich integracji.
- *LLaMA/Ollama – Meta (USA)* – rodzina modeli językowych udostępniana jako open-source, często używana jako podstawa do tworzenia wyspecjalizowanych chatbotów i aplikacji.
- *Bielik – AGH Cyfronet (Polska)* – pierwszy polski duży model językowy trenowany na zasobach w języku polskim, rozwijany w *AGH Cyfronet*.



Rysunek 1.1.: Liczba użytkowników *czatbota* od *OpenAI* (źródło [1])

Jak widać z rysunku 1.1 popularność takich czatbotów stale rośnie.

Pomimo ogromnej wiedzy, jaką takie modele sobą prezentują, nie zawsze są w stanie odpowiedzieć na szczegółowe pytania niezwiązane z wiedzą ogólną, np. inżynierii wodnej. W przypadku braku wiedzy często zmyślają informację i podają ją jako fakty, udając poprawność swoich twierdzeń. Takie zjawisko określane jest mianem halucynacji. Jest ono szczególnie niepożądane w środowisku wymagającym wiedzy wysokospecjalistycznej (np. inżynierii wodnej), gdzie każda decyzja, jaką taka osoba podejmuje, musi być poparta udokumentowaną wiedzą. Każda błędna decyzja może skutkować błędami w wykonywanych projektach, pracach, a w najgorszych przypadkach katastrofami, gdzie może ucierpieć wiele ludzi. Warto jednak też pamiętać, iż osoba decyzyjna, mimo posiadania ogromnej wiedzy, nie jest w stanie zapamiętać różnych niuansów prawnych czy kruczków. Taki inżynier potrzebuje dostępu do systemu *czatbota*, gdzie uzyska odpowiedź popartą dokumentem, w którym informacja jest pewna.

1.2. Cel pracy

Celem tej pracy jest zbadanie potencjału zastosowania metodologii (techniki) RAG do wspierania pracy inżyniera gospodarki wodnej. Osiągnięto to przez połączenie dużego modelu językowego, który generuje odpowiedzi na podstawie bazy danych przechowującej faktyczne informacje. Dzięki temu system najpierw sprawdza prawdziwe źródła, a dopiero potem

odpowiada, co sprawia, że jego odpowiedzi są dokładne i wiarygodne.

System RAG jest projektowany w celu połączenia zdolności do przetwarzania języka naturalnego przez model LLM z funkcjonalnością wyszukiwania danych w istniejących zbiorach informacji. Dzięki temu możliwe jest dostarczanie użytkownikom precyzyjnych i trafnych odpowiedzi opartych na rzeczywistych danych, co czyni ten system idealnym narzędziem w zastosowaniach takich jak analiza dokumentów prawnych, wsparcie procesów decyzyjnych, edukacja czy badania naukowe. Wprowadzenie bazy wektorowej umożliwia skuteczne zarządzanie dużymi ilościami danych, a jednoczesne użycie modelu językowego pozwala na zrozumienie intencji użytkownika oraz generowanie odpowiedzi w naturalnym i czytelnym dla człowieka języku.

1.3. Zakres prac

W ramach pracy wszedł następujący zakres prac:

- Ekstrakcja danych z dokumentów i implementacja ich do bazy danych. Etap polega na przetworzeniu wszystkich danych prawa wodnego jaki i innej literatury i zaimportowaniu ich do bazy dla systemu RAG. Znalezienie modelu osadzeń tekstowych, który odpowiednio zamienia tekst w wektory. Analiza dostępnych rozwiązań baz wektorowych na rynku, wybór najlepszej z dostępnych. Zbadania skuteczności oraz szybkości poszczególnych typów indeksowania takich dokumentów w bazach.

- Stworzenie systemu RAG. Połączenie modelu językowego z pobieraniem dokumentów bazy danych. Implementacja przeróżnych ulepszaczy (wcześniej wspomnianych). Zbadanie poszczególnych dodatkowych metod, sprawdzenie dla których lub której podawane wiadomości są najlepsze. Przegląd dostępnych modeli LLM na rynku. Znalezienie modeli. który najlepiej pasuje do systemu.

- Stworzenie części oceniającej skuteczność systemu. Znalezienie *bibliotek/frameworków* pozwalających na ocenę RAG. Przejrzenie metryk, na których podstawie można ewaluować RAG. Przeprowadzenie badań w celu odnalezienie najlepszej metod ulepszenia systemu uzyskiwania odpowiedzi na temat dokumentu.

2. Przegląd literatury w zakresie Retrieval-Augmented Generation (RAG)

W poniższym rozdziale przedstawiono pozycje naukowe poruszające temat RAG. Przedstawiono pierwsze prototypy tego systemu. Omówiono również sposoby udoskonalenia, poprzez zaprezentowanie modułów w pismach naukowych. Opisano również dokumenty dywagujące na temat ewaluacji takiego systemu.

2.1. Przegląd publikacji

2.1.1. Pierwsze wzmianki o RAG

Jednym z pierwszych znanych artykułów poruszających zagadnienie systemu RAG jest m.in. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* [2]. Zagadnieniem tego tekstu jest problem modeli LLM w precyzyjnym manipulowaniu wiedzą, jaką posiadają, jak i dostępu do niej. W efekcie, w zadaniach sprawdzających te umiejętności nie osiągają najlepszych rezultatów. Dodatkowo dostarczanie dodatkowych informacji poprzez aktualizację ich wiedzy jest problematyczne. Każda dodatkowa informacja, jaką model powinien posiąść, musi być dotrenowana do danego LLMa poprzez fine-tuning.

W związku z zaistniałymi problemami autorzy tekstu proponują rozwiązanie RAG, czyli systemu, w którym model językowy nie bazuje tylko na wiedzy własnej, ale również na wyodrębnionej wiedzy, jaka jest zapisana w wektorowej bazie danych. Składane tam dokumenty są odpowiednio osadzane na wektory, przez co dzięki stworzeniu narzędzia wyszukiwania można zasilić model informacją z poza jego bazy wiedzy.

Warto zauważyć, iż autorzy w architekturze nie stosują LLMa jako modelu generującego odpowiedź. W ich wersji wykorzystany do tego jest model seq2seq. Oryginalnie takie modele sieci neuronowej były wykorzystywane w tłumaczeniach czy streszczeniach tekstów. W tym artykule autorzy pokazują dodatkowe zastosowanie, które w połączeniu z wyszukiwaniem dokumentów tworzy *czatbota*, z precyzyjnymi odpowiedziami.

W dalszej części tekstu autorzy poruszają podstawowe pojęcia związane z systemem RAG. W dokumencie zaprezentowane są także dwie architektury tego rozwiązania:

- RAG model sekwencyjny - system, wykorzystuje dokumenty z bazy danych do generowania oczekiwanej odpowiedzi. Część wyszukująca wiedzę z bazy zwraca najlepsze K pozycji. Następnie są one dostarczane do modelu seq2seq jako jeden pojedynczy tekst i generowana jest odpowiedź.

- RAG model tokenowy - model pobiera dokumenty dla każdego tokena występującego w sekwencji tekstu wejścia. To pozwala generatorowi na wybór konkretnej odpowiedzi z wielu dokumentów. Tak samo jak w poprzednim modelu stosujemy tutaj metodę top K dokumentów. Zatem model pobiera k dokumentów dla każdego tokena z zapytania po czym uśrednia wyniki jaki uzyskał i zwraca odpowiedź.

Twórcy przeprowadzili obszerne eksperymenty na zaimplementowanej architekturze. Zweryfikowali m.in.:

- **Otwarta baza pytań i odpowiedzi** (*Open-domain Question Answering*) – badanie polegało na stworzeniu pary wejście i wyjście. Odpowiednio wejście było zadanym pytaniem a wyjście odpowiedzią. Wcześniej zdefiniowane odpowiedzi do tych pytań pozwoliły autorom na porównaniu wygenerowanej z referencją.

- **Abstrakcyjne pytania i odpowiedzi** (*Abstractive Question Answering*) – testowanie możliwości generowania swobodnych odpowiedzi w zadaniach wymagających intensywnej wiedzy (*knowledge-intensive NLG*). Autorzy wykorzystali zadanie *MSMARCO NLG task v2.1*, gdzie zdefiniowano pytanie tak zwana złota wiedza (*gold passages*) oraz odpowiedź referencyjną. System ma udzielić odpowiedzi na zadane pytania. Mimo posiadania złotej wiedzy nie jest ona używana w procesie tego eksperymentu jedynie badana jest systemowa odpowiedź w porównaniu z referencją.

- **Generowanie pytań w formacie Jeopardy** (*Jeopardy Question Generation*) – zadanie polega na tworzeniu pytań w stylu popularnego Amerykańskiego quizu *Jeopardy*. Uczestnik takie show otrzymuje hasło-odpowiedź, a następnie musi wymyślić do niego odpowiedź. To samo w tym eksperymencie robi model, musi wygenerować pytanie. Ocena obejmowała metrykę *Q-BLEU-1* oraz ewaluacje eksperckie pod kątem faktów i specyficzności.

- **Weryfikacja faktów** (*Fact Verification*) – Zadanie polega na na sklasyfikowaniu twierdzeń jakie stworzył system ze zbiorem danych opartym o *Wikipedię*. W tym eksperymencie badamy poprawność faktów odpowiedzi, weryfikując czy istnieją poparte na jego twierdzenia dokumenty.

Autorzy wprowadzili wiele eksperymentów. W dalszej części podzielili się wynikami tych doświadczeń. Stwierdzono w nich kilka ciekawych spostrzeżeń:

- W testach (*Open-domain Question Answering*) zauważono, że system RAG był w stanie odpowiedzieć poprawnie na pytanie nawet w przypadku gdy w załączonych dokumentach nie jej nie było. Autorzy wypunktowali tutaj wadę innych modeli ekstrakcyjnych, które nie potrafiły by tego dokonać.

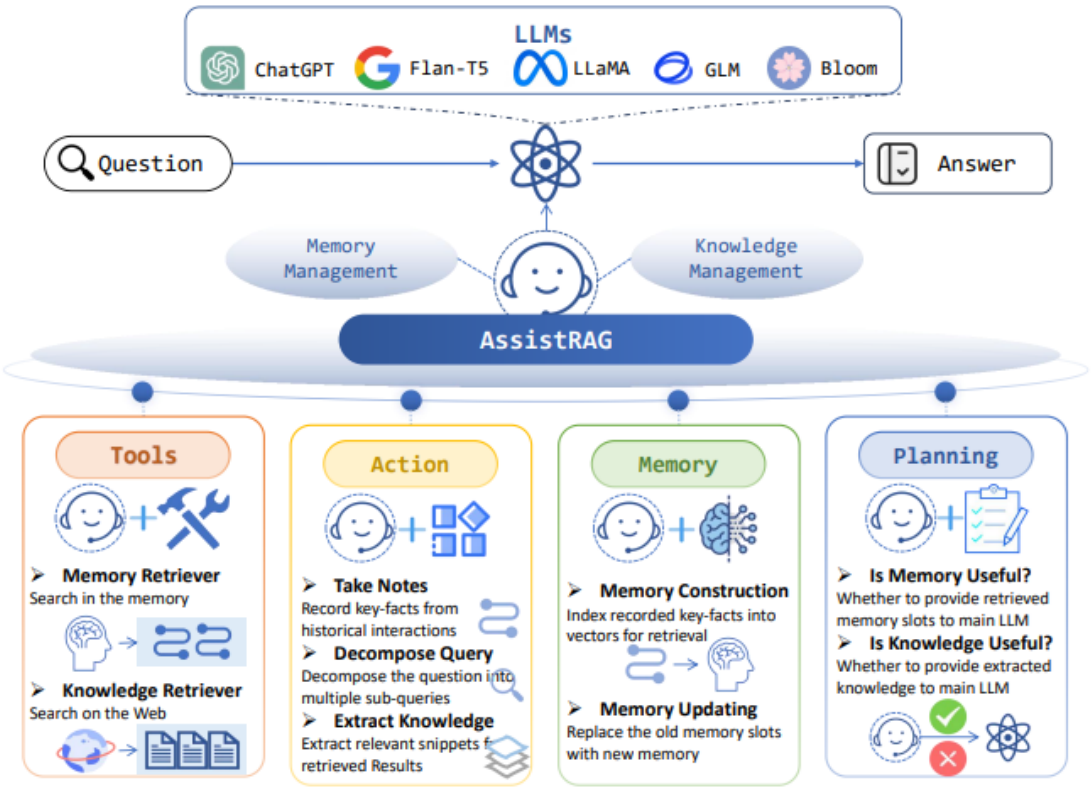
- Podczas badań (*Abstractive Question Answering*) system twórców poradził sobie znacznie lepiej niż porównywalny model BART. Model ten dodatkowo był bardziej podatny na halucynacje w porównaniu z RAGiem.

- W doświadczeniu (*Jeopardy Question Generation*) znowu porównaniu RAG z modelem BART. Oprócz badania rezultatów metrykami wykorzystano tutaj ludzi, którzy sprawdzali odpowiedzi ręcznie i weryfikowali ich poprawność. Większość osób stwierdziło, iż to właśnie RAG poradził sobie z tym zadaniem lepiej.

Podsumowując informacje znajdujące się w tym artykule. Jest to jeden z pierwszych naukowych tekstów, poruszających ten temat. Autorzy zaproponowali architekturę systemu RAG jeszcze na modelach seq2seq czego nie stosuje się obecnie. Dodatkowo przeprowadzone badania odbiegają od tego, co aktualnie się stosuje. To ilustruje szybki rozwój dziedziny przetwarzania języka, gdzie prace naukowe sprzed pięciu lat mogą być już częściowo przestarzałe.

2.1.2. Wzmacnianie systemu

W [3] poruszono temat wzmocnienia RAG poprzez stworzenie różnych dodatkowych modułów. Autorzy tekstu porównywali rozwiązanie podstawowego systemu, gdzie do modelu generującego LLM nie podano nawet promptu z rozwiązaniem promptowym oraz ich autorskim *AssistantRAG*.



Rysunek 2.1.: Schemat AssistantRAG (źródło [3])

Jak można zobaczyć z rysunku 2.1 autorzy wprowadzili wiele ulepszeń do standardowego RAGa wprowadzonego w [2]. Zastosowano tu m.in. LLM jako model generatywny, dodatkowo wprowadzono również pamięć. Pamięć w tym systemie to nic innego jak uzupełnianie bazy wektorowej o nowe informacje, jakie pojawiły się w rozmowie z użytkownikiem, czy to w celu pogłębienia własnej wiedzy, czy też zapisania historii konwersacji. Interesująca jest sekcja 'narzędzi' jaka została zaprezentowana. Narzędzia, jakimi może posługiwać się LLM to nic innego jak napisane w kodzie funkcjonalności, których sam model nie jest w stanie wykorzystać. Przykładowo, takim narzędziem może być dodawanie dwóch liczb, gdzie oryginalnie model przewidziałby wynik, tutaj poda całkowicie poprawny matematycznie. Jest to niezwykle użyteczne w architekturze takiego systemu. Aby model wiedział, że posiada dostępne narzędzia i jak ich używać, należy do każdej takiej funkcji dopisać krótkie streszczenie, co robi. Na jej podstawie LLM jest w stanie wywołać poprawną część kodu.

Ten asystent został również wzbogacony o dostęp do internetu. Model jest w stanie w prosty sposób przebadać temat w wyszukiwarce typu *Google* i zwrócić z tego informacje.

Jest to kolejna rewolucyjna zmiana w porównaniu z oryginalnym rozwiązaniem.

Wnioski płynące z artykułu miały istotne znaczenie dla dalszych prac. W szczególności koncepcja włączania narzędzi do architektury systemu stała się inspiracją do ich późniejszego zastosowania w implementacji systemu.

2.1.3. Ewaluacja modelu RAG

W [4] poruszane są sposoby oraz problemy z ewaluacją systemów RAG. Wyznaczone zostały w nim elementy systemu, jakie należy odpowiednio zbadać:

- Wyszukiwanie (*Retrival*) - W tej części ocenie podlegają zwracane przez system dokumenty. Na podstawie stworzenia par jesteśmy w stanie zbadać metryki:
  - Trafność (*Relevance*) para **(Istotne dokumenty i Pytanie użytkownika)**: badanie pozwala określić jak dobrze pobrane chunki z systemu pasują do zapytania użytkownika. Doświadczenie określa miarę precyzji i specyfikacji procesu wyszukiwania.
  - Dokładność (*Accuracy*) para **(Istotne dokumenty i Zwrócone dokumenty)**: ocenia jak dokładne są dokumenty pobrane w porównaniu z dokumentami kandydatami. Jest to metryka oceniająca umiejętność systemu w zidentyfikowaniu jak najbardziej odpowiednich dokumentów w celu udzielenia odpowiedzi.
- Generowanie (*Generation*) - Podobne relacje dla komponentów generacji przedstawiono poniżej. Skupiono się tutaj na porównaniu odpowiedzi systemu:
  - Trafność (*Relevance*) **Odpowiedź systemu i Pytanie użytkownika**: mierzy jak dobrze wygenerowana odpowiedź jest podobna do pytania użytkownika. Zapewnia aby odpowiedź nawiązywała do tematu zapytania i spełniała wszystkie wymaganie w nim podane.
  - Wierność (*Faithfulness*) **Odpowiedź i Istotne Dokumenty** ocenia, czy wygenerowana odpowiedź dokładnie odzwierciedla informacje zawarte w istotnych dokumentach oraz mierzy spójność pomiędzy wygenerowaną treścią a dokumentami źródłowymi.
  - Poprawność (*Correctness*) **Odpowiedź systemu i Odpowiedź referencyjna** podobnie jak dokładność w komponencie wyszukiwania mierzy poprawność wygenerowanej odpowiedzi w odniesieniu do przykładowej odpowiedzi, która pełni rolę wzorca.

Po przedstawieniu eksperymentów, jakie przeprowadza się na komponentach generowania oraz wyszukiwania, autorzy prezentują dostępne zaimplementowane metody badające te zagadnienia, które zostały zaprezentowane na rysunku 2.2.

Category	Framework	Time	**Raw Targets**	**Retrieval**	**Generation**
Tool	TruEra RAG Triad [54]	2023.10	Context Relevance Answer Relevance *Groundedness*	LLM as a Judge	LLM as a Judge
Tool	LangChain Bench. [32]	2023.11	Accuracy Faithfulness *Execution Time* *Embed. CosDistance*	Accuracy	LLM as a Judge
Tool	Databricks Eval [33]	2023.12	Correctness *Readability* *Comprehensiveness*	-	LLM as a Judge
Benchmark	RAGAs [14]	2023.09	Context Relevance Answer Relevance Faithfulness	LLM as a Judge	LLM Gen + CosSim LLM as a Judge
Benchmark	RECALL [38]	2023.11	Response Quality *Robustness*	-	BLEU, ROUGE-L
Benchmark	ARES [49]	2023.11	Context Relevance Answer Faithfulness Answer Relevance	LLM + Classifier	LLM + Classifier LLM + Classifier
Benchmark	RGB [6]	2023.12	Information Integration *Noise Robustness* *Negative Rejection* *Counterfactual Robustness*	-	Accuracy
Benchmark	MultiHop-RAG [52]	2024.01	Retrieval Quality Response Correctness	MAP, MRR, Hit@K	LLM as a Judge
Benchmark	CRUD-RAG [39]	2024.02	*CREATE*, READ *UPDATE*, *DELETE*	-	ROUGE, BLEU RAGQuestEval
Benchmark	MedRAG [61]	2024.02	Accuracy	-	Accuracy
Benchmark	FeB4RAG [57]	2024.02	Consistency Correctness *Clarity* *Coverage*	-	Human Evaluation Human Evaluation
Benchmark	CDQA [62]	2024.03	Accuracy	-	F1
Benchmark	DomainRAG [58]	2024.06	Correctness Faithfulness *Noise Robustness* *Structural Output*	-	F1, Exact-Match Rouge-L LLM as a Judge
Benchmark	ReEval [66]	2024.06	Hallucination	-	F1, Exacct-Match LLM as a Judge Human Evaluation
Research	FiD-Light [20]	2023.07	*Latency*	-	-
Research	Diversity Reranker [4]	2023.08	*Diversity*	Cosine Distance	-

Rysunek 2.2.: Podział benchmarków (źródło [4])

W późniejszych częściach autorzy dogłębnie tłumaczą, na jakich zbiorach wykonywano ich eksperymenty. Tłumacząc dodatkowo, jak kluczowym w ewaluacji systemu jest odpowiedni dobór takiego zbioru. Poruszają również temat metryk, jakimi można operować w przypadku ewaluacji na tych zbiorach, a były to m.in. *precyzja*, *recall* czy *celność*. Wprowadzone zostają również metryki oparte na rankingu, takie jak: średnia wzajemna ranga (*MRR*) oraz średnia precyzja (*MAP*) pozwalające ewaluować system wyszukiwania. Ciekawym fragmentem

tekstu jest wprowadzenie metryk *ROUGE*, *BLEU* czy *BERT-score*. Nie są to metryki ściśle ewaluujące RAG. *ROUGE* i *BLEU* są używane w przypadku modeli tłumaczenia maszynowego, a *BERT-score* to nic innego jak podobieństwo między dwoma tekstami. Wprowadzenie tych metod jest szczególnie interesujące, bo pozwala na bardziej wszechstronną ocenę jakości generowanych odpowiedzi. Warto jednak pamiętać, iż metryki tłumaczenia maszynowego nie są w stanie zrozumieć synonimów czy zdań brzmiących semantycznie identycznie. Dlatego też autorzy uwzględniają metryki LLMowe. Gdzie dodatkowo implementowany jest model oceniający poprawność działania systemu.

Artykuł pozwolił uporządkować wiedzę na temat metryk stosowanych do oceny systemów. W szczególności zwrócono w nim uwagę na rozróżnienie między metrykami opartymi na modelach językowych a tradycyjnymi metrykami niezależnymi od LLM. W kontekście pracy istotne było podkreślenie, że także klasyczne rozwiązania, takie jak *ROUGE* czy *BLEU*, mogą dostarczać wartościowych informacji o jakości systemu. Z tego powodu metryki te zostały uwzględnione w przeprowadzonych eksperymentach.

2.1.4. Optymalizacja modelu RAG

W [5] porusza się zagadnienie optymalizacji naiwnego RAGa. Omówione są różnorodne metody, które mają potencjał zwiększenia precyzji odpowiedzi dostarczanych przez system. Jedną z metod przedstawionych w artykule jest metoda multi query. Polega ona na wprowadzeniu do systemu dodatkowego punktu przejścia między pobraniem dokumentów z bazy wektorowej a otrzymaniem od użytkownika zagadnienia. Sam proces polega na zwielokrotnieniu zapytania. Wykorzystując model LLM tworzy dodatkowe pytania na podstawie wyjściowego. Dodatkowe pytania często rozwijają temat, który użytkownik poruszył w swoim zapytaniu. Po wprowadzeniu pojęcia autorzy prezentują działanie tej funkcjonalności na kilku przykładach. Zaprezentowane są również metody ewaluacji systemu, takie jak *BERT-score* czy metryki LLMowe oraz *ROUGE*. Autorzy weryfikują działanie kilku wersji systemu, gdzie jednym z kluczowych pojęć jest *indeksowanie z rodzicem*.

Indeksowanie to polega na stworzeniu dwóch typów dokumentów: rodziców i dzieci. Dokumenty dzieci zawierają się w dokumencie rodzicu i są od niego znacząco mniejsze. W przypadku wyszukiwania informacji w bazie danych pobierane są dokumenty dzieci, a dokumenty rodzice są odpowiednio zwracane w etapie generowania.

W celu ewaluacji powstałego systemu autorzy również zastosowali metody, jakie występowały w [4]. Wykorzystanie metryk *ROUGE czy BERT-score* pozwoliło im odpowiednio ocenić rozwiązanie, jakie zaprezentowali. Ostatnimi eksperymentami, jakie zostały przeprowadzane w tym dokumencie, to badanie metrykami LLMowymi.

Na podstawie artykułu zapoznano się z koncepcją multi query, która okazała się szczególnie przydatna w kontekście realizowanej pracy. Mechanizm ten, polegający na generowaniu dodatkowych zapytań w oparciu o zapytanie użytkownika, został następnie wykorzystany w przygotowanej implementacji systemu. Dzięki temu możliwe było zwiększenie precyzji

wyszukiwania dokumentów i poprawa jakości uzyskiwanych odpowiedzi. Zdefiniowany typ *indeksowania z rodzicem* okazał się docelowym rozwiązaniem systemu. Zbadano jego skuteczność w połączeniu z podstawowym rozwiązaniem.

2.2. Podsumowanie

W powyższym rozdziale zostały zaprezentowane dokumenty naukowe na temat RAG. Na łamach tych artykułów oraz książek udało się znaleźć sposoby na rozszerzenie systemu RAG takie jak multi query, czy *indeksowanie z rodzicem*. Dzięki tym metodom stworzono wiele konfiguracji, które kolejno badano na podstawie metryk. Miary, jakie znaleziono w dokumentach, przyczyniły się do zdefiniowania eksperymentów. Dowiedziano się, jakie części systemu RAG należy badać.

Przytoczone narzędzia zostały wykorzystane w implementacji prototypu systemu, istotnie upraszczając proces jego tworzenia i integracji poszczególnych modułów.

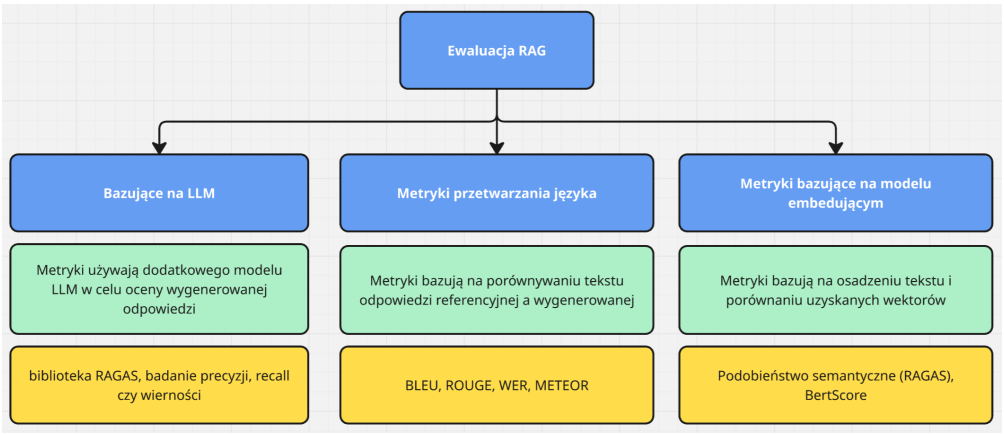
Dodatkowo dzięki [2] przybliżono w małym stopniu najwcześniejsze informacje dotyczące tego systemu.

3. Metodologia badań

Niniejszy rozdział przedstawia wybrane metody i techniki badawcze, które ewaluują system RAG. Zdefiniowano podział metryk na bazujące na LLM, przetwarzaniu języka oraz bazujące na modelu embeddingowym. Szczegółowo opisano każdą z miar, w celu przybliżenia, jaką część systemu bada. Specyfikacja każdej z nich została dodatkowo okraszona odpowiednim wzorem matematycznym. Pełne zrozumienie tego rozdziału pozwoli na poprawną interpretację wyników.

3.1. Badanie działania RAG

Badanie systemów opartych na modelach językowych stanowi wyzwanie, ponieważ trudne jest jednoznaczne ocenienie jakości generowanego przez nie tekstu. Na szczęście dostępne są rozwiązania, które wspierają ten proces. W niniejszej pracy wykorzystano bibliotekę *Ragas* [6], stworzoną specjalnie do analizy systemów typu RAG. Dodatkowo przeprowadzono analizę systemu przy użyciu metryk niezwiązanych bezpośrednio z modelami językowymi. Pełny podział metryk widnieje na poniższym rysunku 3.1.



Rysunek 3.1.: Podział metryk wraz z przykładami.

W celu przeprowadzenia badań przygotowano specjalny zbiór pytań i odpowiedzi opartych na dokumentach, na których operował system. Każde pytanie zostało sporządzone ręcznie, wraz z ręcznie przygotowaną odpowiedzią, zwaną dalej odpowiedzią referencyjną. Badanie

polegało na wprowadzeniu pytania do systemu, a następnie pobraniu dokumentów, na których opierała się generowana przez LLM odpowiedź, oraz samej wygenerowanej odpowiedzi.

3.2. Metryki LLM

3.2.1. Precyzja kontekstu

Precyzja kontekstu to metryka mierząca liczbę istotnych fragmentów (chunków) w pobranych tekstach. Wyróżnia się dwa podejścia do jej obliczania: pierwsze (*precyzja bez referencji*) uwzględnia jedynie zawartość pobraną przez moduł wyszukiwania, natomiast drugie (*precyzja z referencją*) porównuje ją z odpowiedzią referencyjną.

Aby obliczyć metrykę, wykorzystuje się średnią wartość $Precyzji@k$ dla wszystkich fragmentów w kontekście. $Precyzja@k$ definiuje się jako stosunek liczby istotnych dokumentów w top-k do całkowitej liczby fragmentów na poziomie k. Wszystko można opisać wzorem:

$$Precyzja\ Kontekstu@k = \frac{\sum_{k=1}^{K}(precyzja@k\ \times\ v_k)}{liczba\ istotnych\ elementów\ w\ najlepszych\ wynikach\ K} \tag{3.1}$$

$$Precyzja@k = \frac{true\ positives@k}{(true\ positives@k + false\ positives@k)} \tag{3.2}$$

gdzie K jest całkowitą liczbą fragmentów w wygenerowanym kontekście, a $v_k \in \{0, 1\}$ jest wskaźnikiem trafności na poziomie rangi.

Metryka ta bada działanie modułu wyszukiwania w systemie RAG. Sprawdzane jest, jak dostarczone dokumenty wpływają na jakość generowanych odpowiedzi oraz trafność informacji zwracanych przez model.

3.2.2. Recall kontekstu

Recall kontekstu mierzy liczbę istotnych dokumentów zwróconych przez RAG. Koncentruje się on na tym, aby nie pominąć ważnych danych. Im wyższy wynik, tym mniejsza ilość ważnego tekstu została pominięta w udzieleniu odpowiedzi. Decydującym elementem tej metryki jest oceniający model LLM, który na podstawie odpowiedzi referencyjnej oraz pobranych dokumentów wyodrębnia twierdzenia, z których następnie obliczana jest wartość metryki.

W celu obliczenia *recallu kontekstu* wykorzystano wzór:

$$Recall\ kontekstu\ = \frac{Liczba\ twierdzeń\ w\ odpowiedzi\ referencyjnej\ popartych\ kontekstami}{Całkowita\ liczba\ twierdzeń\ w\ odpowiedzi\ referencyjnej} \tag{3.3}$$

Metryka ponownie koncentruje się na etapie pobierania dokumentów z bazy danych, sprawdzając, czy dostarczone dokumenty zawierają wystarczającą liczbę twierdzeń potrzebnych do stworzenia wyczerpującej odpowiedzi.

3.2.3. Recall encji

Jest to rozszerzenie *recallu kontekstu*. Metryka bazuje na przetworzeniu pobranych dokumentów oraz odpowiedzi referencyjnej w celu wyodrębnienia z nich encji, czyli konkretnych jednostek informacji, takich jak nazwy osób, miejsc, organizacji, dat czy innych kluczowych pojęć, które mają znaczenie dla odpowiedzi.

- Zbiór RE: zbiór encji z tekstów odniesienia.
- Zbiór RCE: zbiór encji z tekstów pobranych przez system.

Następnie rezultat jest wyliczany na podstawie wzoru:

$$recall\ encji = \frac{RE \cap RCE}{RE} \tag{3.4}$$

Wynik jest zwracany z przedziału $[0, 1]$. Im wyższa liczba, tym większa część encji z odniesienia zostaje uwzględniona przez RAG. Metryka recallu encji pozwala nie tylko ocenić skuteczność samego mechanizmu pobierania dokumentów, ale także sprawdzić, czy system jest w stanie wychwycić kluczowe jednostki informacji.

3.2.4. Wrażliwość na szum

Jest to metryka mierząca, jak źle poradził sobie system z odpowiadaniem. Zatem wymagany jest tutaj jak najniższy wynik. Aby wyliczyć wrażliwość na szum, odpowiedź udzieloną przez model jest analizowana pod kątem występujących w nich twierdzeń. Sprawdzane jest, czy twierdzenia, które uzyskano z analizy odpowiedzi, występują w odpowiedzi referencyjnej lub czy znajdują się w jakimkolwiek fragmencie tekstu zwróconym przez system RAG. Idealnie byłoby, gdyby wszystkie twierdzenia w odpowiedzi były poparte odpowiednim uzyskanym dokumentem. Całość metryki można określić następującym wzorem:

$$Wrażliwość\ na\ szum = \frac{|Łączna\ liczba\ niepoprawnych\ twierdzeń\ w\ odpowiedzi|}{|Łączna\ liczba\ twierdzeń\ w\ odpowiedzi|} \tag{3.5}$$

3.2.5. Wierność

Wierność jest metryką badającą, jak mocno model halucynuje, udzielając odpowiedzi. Do zbadania tego używany jest model *Vectara's HHEM-2.1-Open* [7] oparty na architekturze T5. Model ten jest wykorzystywany w drugim etapie obliczania metryki wierności, tj.

podczas weryfikacji, czy dane twierdzenia można wywnioskować z dostarczonego kontekstu. Oprócz wcześniej wspomnianego modelu w metryce potrzebny jest drugi model, językowy, który dokonuje weryfikacji w pierwszym etapie. Warto tutaj zaznaczyć, iż metryka nie bada, czy w odpowiedzi znajdują się wszystkie ważne informacje. Dzięki niej sprawdzane jest, czy system nie kłamie użytkownikowi na dany temat, wymyślając niepoprawne fakty (fakt to stwierdzenie w odpowiedzi uznane przez model za prawdę, np. Polska odzyskała niepodległość w 1918, tutaj faktem może być odzyskanie niepodległości czy rok, w którym to się odbyło). Ostatecznie metrykę można opisać wzorem:

$$Wierność = \frac{Ilość\ faktów\ w\ odpowiedzi\ potwierdzonych\ w\ pobranych\ dokumentach}{Całkowita\ ilość\ faktów\ w\ odpowiedzi} \tag{3.6}$$

3.2.6. Zgodność z faktami

Metryka ta ocenia zgodność faktów zawartych w wygenerowanej odpowiedzi względem odpowiedzi referencyjnej. Jest używana do oceny zgodności między odpowiedzią wygenerowaną a referencyjną. Wynik tej metryki zawiera się w zbiorze od 0 do 1, gdzie im wyższa wartość, tym lepiej. Do wykonania pomiarów metryka używa ewaluacyjnego modelu językowego. W celu obliczenia rezultatu model najpierw dzieli odpowiedź i odniesienie kolejno na twierdzenia, jakie w nich występują. Następnie wykorzystuje wnioskowanie w języku naturalnym, aby określić faktyczne nakładanie (*factual overlap*) się odpowiedzi i odniesienia. Faktyczne nakładanie się jest kwantyfikowane za pomocą *precyzji*, *recall* i wyniku *F1*.

3.3. Metryki przetwarzania języka

W badaniach skorzystano również z biblioteki *RapidFuzz* [8], która umożliwia ocenę podobieństwa tekstów bez użycia modeli LLM. W tym przypadku zastosowano dodatkowe metryki porównywania tekstu, powszechnie wykorzystywane w przetwarzaniu języka naturalnego:

- *Odległości Levenshteina* - metryka mówi, iż odległość pomiędzy dwoma napisami jest najmniejsza kiedy liczba działań prostych jest najmniejsza. Działania proste to kroki przeprowadzających jeden napis na drugi poprzez dodawanie, zamianę lub usuwanie znaków.

- *Odległości Hamminga* - miara odmienności dwóch ciągów o takiej samej długości, wyrażająca liczbę miejsc (pozycji), na których te dwa ciągi się różnią. Innymi słowy jest to najmniejsza liczba zmian (operacji zastępowania elementu innym), jakie pozwalają przeprowadzić jeden ciąg na drugi. Z powodu warunku o tej samej długości obu tekstów nie zakłada się dobrych wyników.

- *Odległości Jaro* - tą miarę dla słów s_1 i s_2 liczy się zgodnie ze wzorem:

$$Jaro(s_1, s_2) = \begin{cases} 0, \ jeli & dla\ m = 0 \\ \frac{1}{3}(\frac{m}{|s_1|} + \frac{m}{|s_2|} + \frac{m-t}{m}) & dla\ m \neq 0 \end{cases} \tag{3.7}$$

  gdzie m to liczba pasujących znaków, a t to liczba transpozycji. Odpowiednio
  - m jest wyliczane na podstawie ilości takich samych znaków w obu słowach jakie są w odległości od siebie maksymalnie równej $\left\lfloor \frac{max(|s_1|,|s_2|)}{2} \right\rfloor$.
  - t czyli liczba transpozycji określa się równą połowie liczby znaków dopasowanych, które występują w innej kolejności.

  Zatem dla przykładu w słowach *PIES* i *PIEC* liczba dopasowań liter będzie wynosić 3 (to będzie wartość m) a liczba transpozycji 0 (to jest wartość t), ponieważ nie można zmienić kolejności znaków w taki sposób aby stworzyć te same wyrazy, gdyż $C \neq S$ a w podciągu PIE wszystkie znaki są takie same. Ostatecznie wynik tej miary będzie wynosił:

$$Jaro(PIES, PIEC) = \frac{1}{3}(\frac{3}{|4|} + \frac{3}{|4|} + \frac{3-0}{3}) = \frac{5}{6} \approx 0.8333 \tag{3.8}$$

- Odległości Jaro-Winkler - ta metryka jest rozszerzeniem wyżej wyjaśnione metryki Jaro. Liczy się ją na podstawie wzoru:

$$JaroWrinkler(s_1, s_2) = Jaro(s_1, s_2) + l \cdot p \cdot (1 - Jaro(s_1, s_2)) \tag{3.9}$$

  gdzie odpowiednio:
  - l jest długością wspólnego prefiksu (max 4 znaki).
  - p jest stałą zazwyczaj równą 0.1.

  Wyliczając wartość *Jaro-Winklera* dla wcześniejszego przykładu $PIES$ i $PIEC$ otrzymano:

$$\begin{aligned} JaroWrinkler(PIES, PIEC) &= Jaro(PIES, PIEC) + l \cdot p \cdot (1 - Jaro(PIES, PIEC)) \\ JaroWrinkler(PIES, PIEC) &= Jaro(PIES, PIEC) + 3 \cdot 0.1 \cdot (1 - Jaro(PIES, PIEC)) \\ JaroWrinkler(PIES, PIEC) &= \frac{5}{6} + \frac{3}{10} \cdot \frac{1}{6} \approx 0.8333 \end{aligned} \tag{3.10}$$

3.3.1. BLEU

Następne dwie metryki opisane w tym podrozdziale nie są metrykami ściśle do ewaluowania systemu RAG, ich głównym zastosowaniem jest ocena modeli tłumaczących. Jedną z nich

jest właśnie *BLEU*, czyli *Dwujęzyczny Asystent Oceny* (*Bilingual Evaluation Understudy*). Bazuje on na tworzeniu tak zwanych n-gramów, czyli połączeń słów w tekście. Przykładowo, n-gramy drugiego rzędu ze zdania „Ala ma kota" to: „Ala ma" i „ma kota". Kolejność słów odgrywa tu kluczową rolę i nie generuje się wszystkich możliwych permutacji ich wystąpień. W przypadku tłumaczeń szyk wyrazów jest szczególnie istotny, natomiast w RAG nie jest aż tak krytyczny, choć nie można go całkowicie ignorować. Matematycznie metrykę można zapisać:

$$BLEU = BP * exp(\sum_{n=1}^{N}(w_n \cdot ln(p_n)) \tag{3.11}$$

gdzie odpowiednio:

- p_i czyli zmodyfikowana precyzja n-gramów można ją opisać wzorem:

$$p_n = \frac{min(dopasowania_n, maks.\ liczba\ referencji)}{ilość\ wystąpień\ wszystkich\ n\text{-}gramów\ w\ wygenerowanym\ tekście} \tag{3.12}$$

  Przy czym:

  - $dopasowania_n$ jest liczbą n-gramów rzędu n, które dokładnie odpowiadają tekstu wygenerowanemu i dowolnemu z referencyjnych.
  - $maks.\ liczba\ referencji$ jest maksymalną liczbą wystąpień określonego n-gramu rzędu n znalezionego w dowolnym pojedynczym referencyjnym tekście.

- BP oznacza karę za zwięzłość (*Brevity Penalty*). Kara za zwięzłość nakłada karę na wygenerowane teksty krótsze od referencyjnych wersji. Wzór matematyczny jest następujący:

$$BP = exp(1 - \frac{r}{c}) \tag{3.13}$$

  gdzie:

  - r jest długością tekstu wygenerowanego.
  - c jest średnią długością tekstów referencyjnych.

- w_n jest wagą dla poszczególnych n-gramów w naszych badaniach zastosowano wagę $\frac{1}{4}$ dla pierwszych trzech n-gramów i 0 dla ostatniego.

- N jest ilością n-gramów w przypadku tej pracy wynoszącym 3, czyli będą realizowane maksymalnie sklejenia 3 wyrazów.

Podsumowując, metryka *BLUE* jest dobrym rozwiązaniem nawet dla systemów RAGowych. Można użyć jej do oceny dokładności odpowiedzi systemu poprzez porównanie słów między odpowiedzią a referencją. Warto jednak zaznaczyć, iż w przypadku zdań identycznych semantycznie zwróci ona zakłamany wynik. Dlatego też nie powinna być używana jako główne źródło badań.

3.3.2. ROUGE

ROUGE, czy też *ocena streszczeń ukierunkowanych na recall* (*Recall-Oriented Understudy for Gisting Evaluation*) jest ostatnią metryką, z jaką prowadzono badania. Jak sama nazwa wskazuje, metryka kładzie główny nacisk na badanie wartości *recall*, jednak nie oznacza to, że pomija inne miary. Uwzględnia również m.in. *precyzję* oraz *F1*. Identycznie jak w *BLEU*, używane są tutaj n-gramy, tylko tym razem do drugiego rzędu ($ROUGE_1$ i $ROUGE_2$) oraz korzysta się z najdłuższego podciągu ($ROUGE_L$). Odpowiednio można wyznaczyć wzory na wartości zwracane przez ROUGE:

- $$ROUGE_n\ recall = \frac{liczba\ wspólnych\ n\text{-}gramów}{liczba\ n\text{-}gramów\ w\ referencji} \tag{3.14}$$
- $$ROUGE_n\ precision = \frac{liczba\ wspólnych\ n\text{-}gramów}{liczba\ n\text{-}gramów\ w\ wygenerowanym\ tekście} \tag{3.15}$$
- $$ROUGE_n\ F1 = \frac{2 \cdot ROUGE_n\ precision \cdot ROUGE_n\ recall}{ROUGE_n\ precision + ROUGE_n\ recall} \tag{3.16}$$

Podobnie jak *BLEU*, *ROUGE* może niesprawiedliwie oceniać system RAG, który generuje semantycznie poprawne zdania, lecz z innym szykiem lub słownictwem w porównaniu z referencją.

3.4. Metryki z modelem embeddującym

3.4.1. Podobieństwo semantyczne

Jest to metryka mierząca podobieństwo między odpowiedzią udzieloną przez system a odniesieniem przygotowanym ręcznie. Badanie polega na osadzeniu odpowiedzi, jak i referencji, za pomocą modelu embeddującego i porównaniu wygenerowanych przez niego wektorów. Wyższy wynik oznacza lepszą zgodność wygenerowanej odpowiedzi z prawdą.

3.4.2. Trafność odpowiedzi

Trafność odpowiedzi to metryka oparta na ocenie przez LLM z dodatkiem modelu embeddującego, gdzie badane jest, jak dobrze dana odpowiedź wpasowuje się w pytanie. Sprawdzane jest, czy odpowiedź zawiera wszystkie istotne informacje oraz czy nie zawiera zbędnego tekstu. Metryka jest liczona w oparciu o pytania użytkownika i odpowiedzi:

- Najpierw generowane są pytania na podstawie odpowiedzi. Pytania te mają odzwierciedlać jej treść.

- Później, zarówno wygenerowane pytania (E_{g_i}), jak i pytanie użytkownika (E_0) zostają przekształcone na wektory, po czym oblicza się między nimi *odległość kosinusową*.
- Następnie w celu obliczenia metryki brany jest średni wynik dystansu między innymi pytaniami a oryginałem. Można to opisać wzorem:

$$\text{Trafność odpowiedzi} = \frac{1}{N}\sum_{n=1}^{N}\frac{E_{g_i}\cdot E_0}{\|E_{g_i}\|\|E_0\|} \tag{3.17}$$

Podsumowując, metryka nie bada, czy odpowiedź jest poprawna. Sprawdzane jest, jak dobrze odpowiedź zawiera wszystkie tematy związane z pytaniem. Trafność karze za niekompletne odpowiedzi lub odpowiedzi posiadające zbędne detale.

3.5. Podsumowanie

Przedstawiona metodologia dostarcza wszechstronnych informacji na temat realizacji badań eksperymentalnych z wykorzystaniem systemu RAG. Zdefiniowano podział miar na rodzaj bazujących na modelu LLM, używających modelu embeddującego oraz na metryki posługujące się odległościami używanymi w przetwarzaniu języka. Przedstawione miary badają każdy aspekt systemu od momentu pobrania danych z bazy wektorowej do generowania odpowiedzi. To szczególnie ważne, ponieważ ocenie podlega cały system, co pozwala zidentyfikować miejsca, w których występują braki.

Dzięki zastosowaniu tak szerokiego przedziału metryk możliwe jest porównanie wariantów systemu oraz identyfikacja czynników mających największy wpływ na poprawność odpowiedzi.

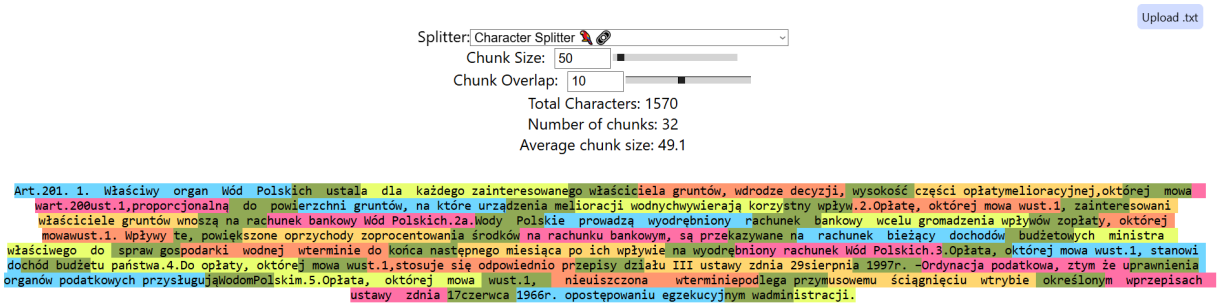
Poprawna interpretacja wyników metryk jest kluczowa dla wyciągania rzetelnych wniosków dotyczących skuteczności i ograniczeń systemu RAG.

4. Koncepcja architektoniczna rozwiązania

W niniejszym rozdziale zostanie przedstawiona architektura zaimplementowanego systemu RAG. Zostanie wytłumaczone działanie poszczególnych części systemu oraz przedstawione, dlaczego to one zostały wybrane w finalnej wersji w celu przeprowadzenia eksperymentów. Zaprezentowane zostaną schematy narzędzi oraz polecenia dla modelu LLM na jakich działa system. System posiada swoją nazwę, która będzie używana podczas odniesień: ***AquaRag****.*

4.1. Proces segmentacji tekstu

Najważniejszym elementem działania koncepcji RAG jest proces segmentacji. To właśnie tutaj dzielony jest tekst na odpowiednie fragmenty, które później trafiają do modelu językowego w celu utworzenia odpowiedzi. Dzielenie tekstu polega na niczym innym, jak rozczłonkowaniu źródłowego tekstu na pomniejsze fragmenty z informacjami, tak zwane chunki. Aby to uzyskać w bibliotece *langchain* [9] zaimplementowano klasę *CharacterTextSplitter*. Z jej pomocą można zdefragmentować tekst; niestety nie uwzględnia ona struktury dzielonego dokumentu, tj. dokonuje ona fragmentacji na podstawie wcześniej zdefiniowanej długości, na jaką chcemy podzielić tekst (długość chunków). Z tego powodu w projekcie badawczym użyto klasy: *RecursiveCharacterTextSplitter*. Dzięki niej można segmentować, uwzględniając akapity, kropki, przecinki, nowe linie lub własne zadeklarowane punkty separacji. W przypadku tekstu Prawa Wodnego było to szczególnie przydatne, bo zfragmentowano tekst na podstawie początków każdego z artykułów. Dodatkowo, podczas tworzenia fragmentów tekstu, ważne jest użycie funkcji overlappingu, która pozwala zazębić chunki.



Rysunek 4.1.: Przykład overlappingu dla artykułu 201 Prawa wodnego (źródło [10])

Jak widać z rysunku 4.1 podzielony tekst posiada odcinki wspólne dla każdego z fragmentów (zaznaczone na zielono). Celem stosowania tej techniki jest zachowanie pełnego kontekstu analizowanych fragmentów. Na przykład możliwe jest znalezienie fragmentu tekstu odpowiadającego na konkretne pytanie, jednak pełna myśl może być rozwinięta, obejmując inne fragmenty. Dzięki zastosowaniu overlappingu możliwe jest kontrolowanie procesu tak, aby cała myśl była zwrócona w sposób kompletny, poprzez zwiększenie prawdopodobieństwa pobrania kolejnej części myśli.

4.2. Osadzenie i indeksowanie podzielonych fragmentów

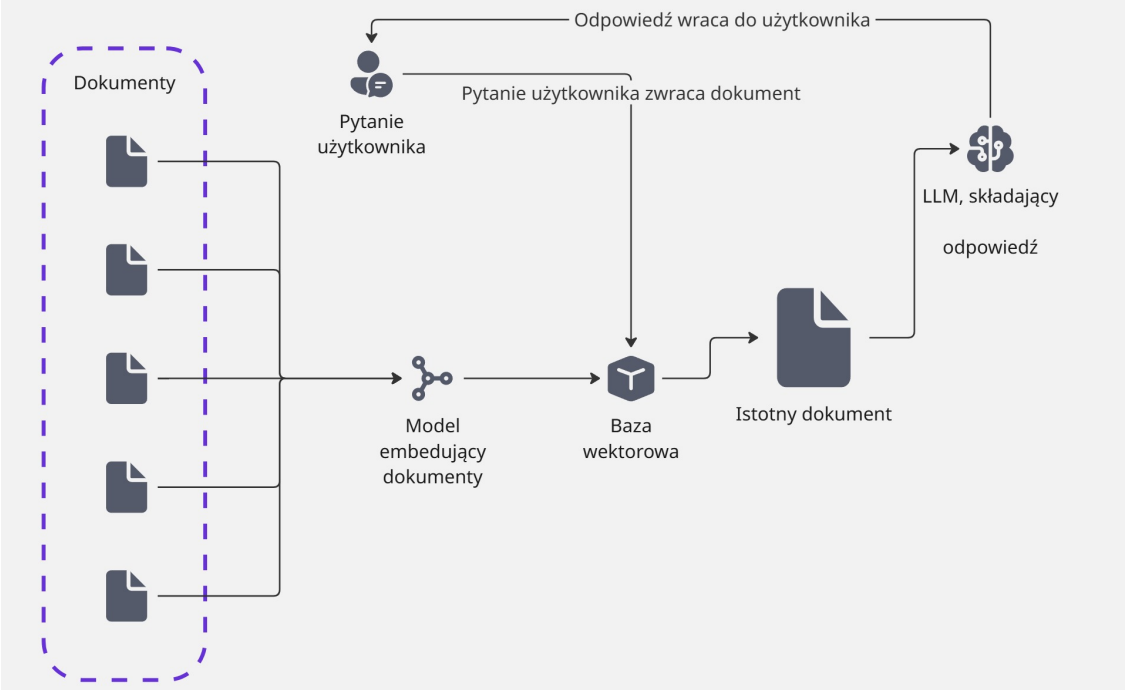
Osadzenie to sposób na zamianę podanego tekstu na wektor. Do tego celu używa się specjalnych modeli do embeddingu, które przyjmują na wejściu tekst i zwracają wektor. Długość takiego wektora jest całkowicie zależna od modelu. Podczas badań przetestowano kilka takich modeli. Użycie ich jest w pełni darmowe, a dokładność, z jaką później baza danych zwraca dokumenty, jest na wysokim poziomie. Ostatecznie wybrano model *nvidia/nv-embed-v1* [11], który zwraca wektory o rozmiarze 4096.

Istnieje wiele sposobów indeksowania już podzielonego tekstu źródłowego po osadzeniu, takie jak **wykorzystanie samego podziału tekstu**, metoda **wyszukiwania z rodzicem** czy podejście **multi-wektor**. W ramach przeprowadzonych badań przetestowano dwie z wymienionych metod: pierwszą i drugą.

Jednym z problemów indeksowania dokumentów jest brak kontekstu lub jego zbyt duża ilość, jaką fragmenty posiadają. W przypadku dużych chunków posiadany jest kontekst całej odpowiedzi, ale z tego powodu trudniej trafić w dokument podczas wyszukiwania z bazy. Dzieje się tak z powodu wielu treści poruszanych w fragmencie, przez co wygenerowany wektor jest różny od zapytania. W przypadku małych fragmentów istnieje większa celność trafienia w wymagany temat, jednak podczas składania wypowiedzi może brakować kluczowych informacji. Każda z niżej wymienionych metod stara się wyeliminować te problemy na

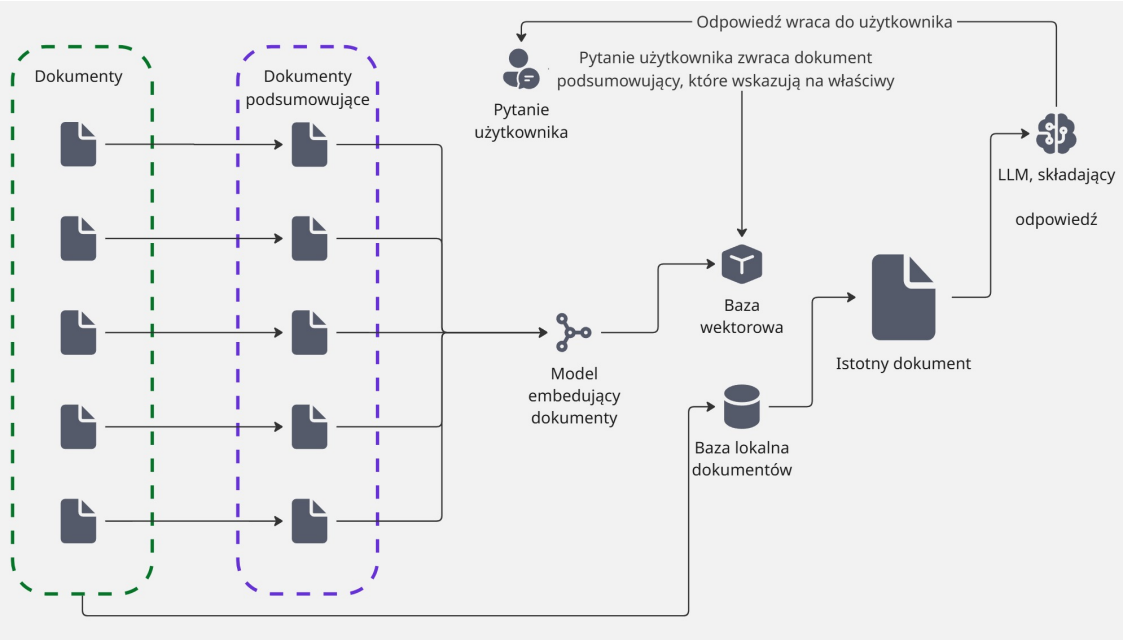
swój sposób.

- **Użycie podstawowego podziału tekstu** - metoda polega na zamianie w wektor podstawowych fragmentów i późniejszym ich wyszukiwaniu w bazie danych. Jest to bardzo podstawowy sposób, który liczy, że wymieniony wyżej problem się nie pojawi. Pełny schemat został zaprezentowany na rysunku 4.2.



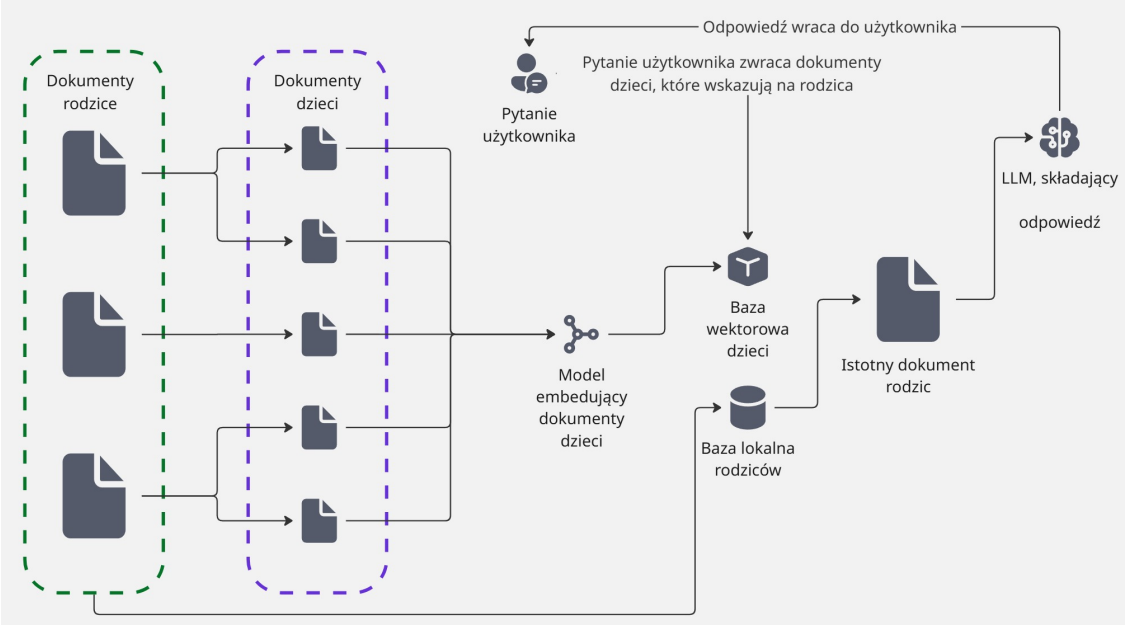
Rysunek 4.2.: Indeksowanie podstawowe

- **multi-wektor** - metoda bazuje na dodatkowym wykorzystaniu modelu językowego podczas dodawania dokumentów do bazy danych. Zamiast dodawania "suchego" dokumentu wcześniej jest on przetwarzany przez model, który streszcza jego tekst. Następnie do embeddingu wykorzystywane jest jego podsumowanie. Dzięki tej metodzie rośnie szansa na trafienie w wymagany chunk, ponieważ, wektor nie zawiera zbędnego szumu, który występowałby w całym fragmencie. Dodatkowo, podczas zwracania pełnego tekstu, zapewniany jest wymagany kontekst. Wadą tego podejścia jest jednak konieczność dodatkowego wykorzystania modelu, co w przypadku rozwiązań płatnych generuje znacząco wyższe koszty niż w innych opisanych rozwiązaniach.



Rysunek 4.3.: Indeksowanie z multi-wektorem

- **Wyszukiwanie z rodzicem** - rozwiązanie opiera się na stworzeniu dwóch odrębnych separatorów tekstów. Jeden dzieli tekst na duże fragmenty tak zwanych rodziców, natomiast drugi dzieli na małe fragmenty zwane dziećmi. Dzieci zawierają się w rodzicach i nie mogą po za nie wybiegać. W następnym kroku są zamieniane na wektor oraz dodawane do bazy danych. To właśnie na nich szukane są dokumenty potrzebne do udzielenia odpowiedzi, jednakże to nie one będą zwracane do generatora. Każde dziecko posiada odnośnik do swojego rodzica i w momencie kiedy system tworzy odpowiedź, to generator otrzymuje przypisanych mu rodziców. Rodzice zapisywani są lokalnie w postaci plików opatrzonych odpowiednim identyfikatorem. Takie rozwiązanie zapewnia lepsze dopasowanie pytania do fragmentu tekstu (dzieci) nie martwiąc się o utratę kontekstu odpowiednio zwracając większą partię tekstu. W przypadku Prawa Wodnego artykuły często zawierają różne podsekcje, które idealnie nadawały się na dokumentny dzieci, gdzie cały artykuł zostawał rodzicem. Pełny schemat został zaprezentowany na rysunku 4.4.



Rysunek 4.4.: Indeksowanie z rodzicem

4.3. Baza danych wektorowych

4.3.1. Wprowadzenie

Najważniejszą częścią każdego systemu typu RAG jest pozyskanie odpowiedniego kontekstu dotyczącego pytania, w tym celu dokumenty zapisywane są w wektorowej bazie danych. Silnik bazy danych umożliwia znalezienie odpowiedniego dokumentu, poprzez policzenie podobieństwa odpowiednią odległością. W tym celu, podczas tworzenia kolekcji dokumentów, definiowana jest metryka, na podstawie której będą zwracane dokumenty, a może to być m.in.

- *odległość kosinusowa* - mierzy kąt pomiędzy wektorami w przestrzeni wielowymiarowej, nie uwzględniając ich długości.
- *iloczyn skalarny* - porównuje wektory poprzez sumowanie iloczynów odpowiadających sobie współrzędnych.
- *odległość euklidesowa* - klasyczna miara geometryczna w przestrzeni wielowymiarowej, mierzącą faktyczną długość odcinka między punktami (wektorami).

Kluczowym w systemie jest znalezienie metryki odpowiadającej naszemu dokumentowi źródłowemu. W przypadku badań wybrano standardową *odległość kosinusową*.

4.3.2. Dostępne silniki baz danych na rynku

Zapotrzebowanie na rozwiązania wektorowej bazy danych ciągle rośnie, dlatego dostępnych opcji na rynku jest wiele. Podczas preselekcji danej bazy kierowano się kilkoma podpunktami, które były istotne.

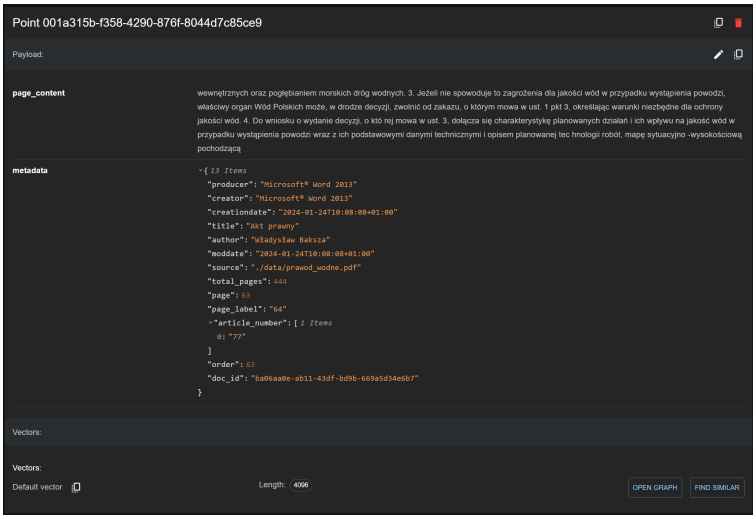
- Dostępne metryki: ważne było aby były dostępne trzy wcześniej wymienione metryki
- Możliwość przeniesienia rozwiązania do chmury
- Szybkość zwracanych dokumentów

Po sprawdzeniu aktualnych rozwiązań udało się wyselekcjonować trzy silniki, są to: *Qdrant* [12], *ChromaDB* [13], *pgvector* [14] (rozszerzenie *PostgreSQL*).

4.3.3. *Qdrant*

To jedna z najpopularniejszych baz danych dostępnych na rynku. Korzystając z niej, otrzymuje się również dostęp do pulpitu nawigacyjnego, który umożliwia samodzielne przeszukiwanie zgromadzonych danych. Pozwala to na szybkie sprawdzenie, jakie dokumenty są

zwracane dla danego zapytania. Niestety wcześniej takie pytanie musimy odpowiednio zamienić na wektor, ponieważ baza udostępnia tylko wyszukiwanie po wektorach, nie posiada wbudowanego modelu do embeddingu.



Rysunek 4.5.: Pojedynczy rekord bazy *Qdrant*

Dodatkowo *Qdrant* gwarantuje nam możliwość dodania własnych metadanych do każdego z wpisów. W przypadku dokumentu Prawa Wodnego okazało się to niezwykle przydatne. Do każdego fragmentu został dodany nr artykułu czy nr strony, z której pochodzi fragment. Dzięki temu stworzono dodatkowe informacje, które mogą okazać się przydatne podczas generowania odpowiedzi.

Jak widać z rysunku 4.5 każdy z punktów posiada wspomniane metadane. W tym przypadku jest to tytuł, autor książki czy nr artykułu/artykułów, z jakich pochodzi tekst. Na rysunku jest zamieszczony dokument dziecko z rozwiązania rodziców i dzieci, zatem w metadanych posiada on również odnośnik do dokumentu rodzica (*doc_id*).

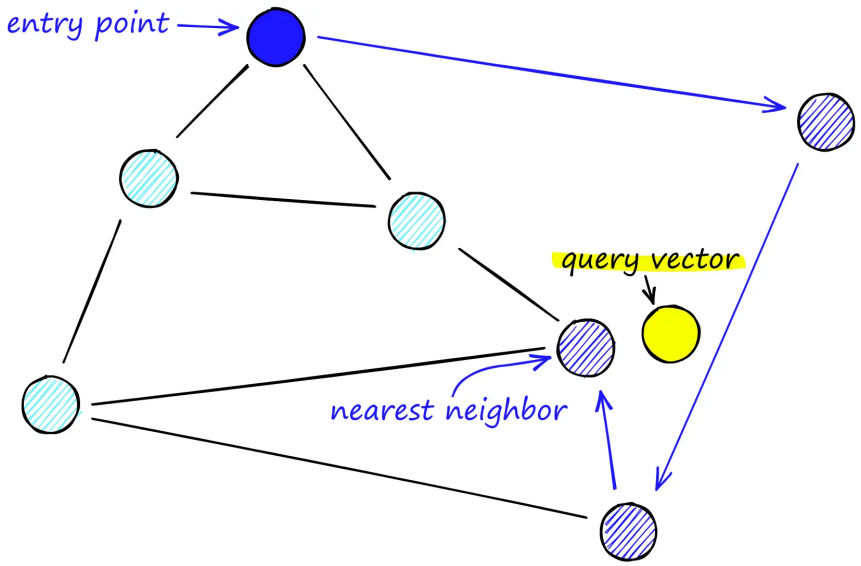
Zanim zostanie wyjaśniony sposób pobierania danych z bazy, warto wpierw omówić sposób przechowywania wektorów. Pojedynczy rekord został przedstawiony na rysunku 4.5 jak widać, składa się on z trzech części.

- kontent strony (*page_content*) - jest to fragment tekstu jaki został wyodrębniony podczas segmentowania.
- metadane - wcześniej wspomniane dodatkowe informacje potrzebne w dalszym przetwarzaniu lub filtrowaniu
- wektor - wektor o wymiarze x, zależnym od modele osadzającego tekst

Każdy z takich rekordów znajduje się w swojej kolekcji w bazie. Kolekcję z *Qdrant* można traktować tak naprawdę jako tabelę, gdzie najważniejszą wartością jest wektor.

Qdrant używa algorytmu *Hierarchical navigable small world (HNSW)* w celu odnajdywania odpowiednich wektorów. Aby wyjaśnić działania tej metody, należy wcześniej przytoczyć dwie mniejsze struktury, z których się składa:

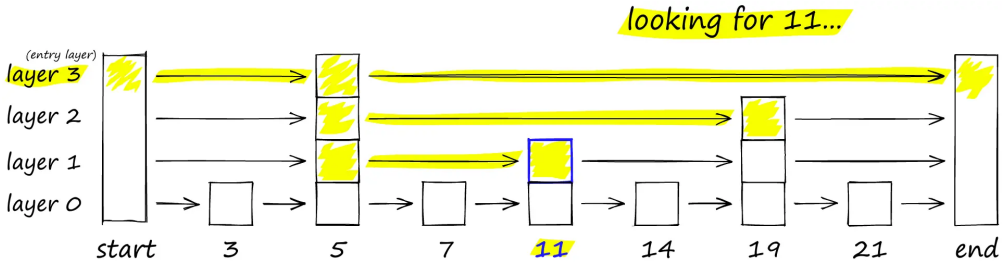
- *Navigable Small World Graphs* (część *NSW* z *HNSW*) - jest to algorytm przeszukiwania zbioru wektorów poprzez tworzenie z nich grafu. Polega na łączeniu wektorów w taki sposób, aby każda krawędź prowadziła do najbliższego jego sąsiada. W praktyce dla każdego wektora tworzy się k takich krawędzi znajdując k sąsiadów. Po połączeniu wszystkich wektorów powstaje graf o maksymalnym stopniu wierzchołka równym k. Dzięki temu, po wprowadzeniu nowego wektora, którego najbliższe odpowiedniki chcemy znaleźć, wystarczy przejść przez graf, aby odnaleźć jego najbliższych sąsiadów.



Rysunek 4.6.: Algorytm *NSW* (źródło [15])

Na rysunku 4.6 zaprezentowano przykładowe wyszukiwanie najbliższych sąsiadów za pomocą algorytmu *NSW*.

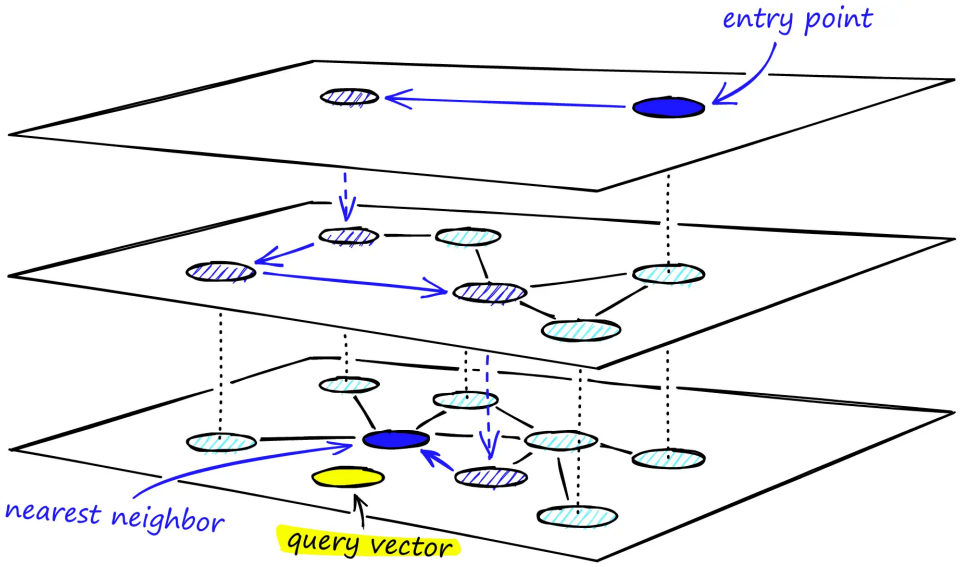
- *Lista z przeskokami* (część *H* z *HNSW*) - jest to struktura bazująca na liście (*linked list*), która posiada poziomy hierarchii. Dolny poziom to pełna lista z wszystkimi elementami, natomiast wyższy poziom posiada przeskoki, czyli pominięcia kilku elementów z oryginalnej listy np. co drugi. Zatem proces znajdowania pozycji z takiej struktury bazuje na znalezieniu przedziału z wyższych poziomów i stopniowym zniżaniu się do pełnej listy w celu namierzenia szukanego elementu.



Rysunek 4.7.: Lista z przeskokami (źródło [15])

Na rysunku 4.7 zademonstrowano przeszukiwanie *listy z przeskokami*, gdzie poszukiwanym elementem jest liczba 11.

Bazując na wyżej wymienionych strukturach, powstaje algorytm *HNSW*, w którym wyszukiwanie najbliższych wierzchołków odbywa się na tej samej zasadzie, co w liście z przeskokami, sprawdzając kolejno każdy poziom hierarchii. Algorytm budowania poszczególnych poziomów działa natomiast na zasadzie tworzenia grafu zgodnie z metodą *NSW*.



Rysunek 4.8.: Algorytm *HNSW* (źródło [15])

Na rysunku 4.8 przedstawiono działanie algorytmu *HNSW*. Wpierw sprawdzany jest pierwszy poziom i najbliższy sąsiad poszukiwanego wektora. Następnie, schodząc stopniowo na ko-

lejne poziomy hierarchii, weryfikowany jest najbliższy sąsiad na danym poziomie. Po zejściu na najniższy stopień hierarchii i zweryfikowaniu wyniku, algorytm zwraca sąsiadów.

4.3.4. *ChromaDB*

ChromaDB jest oryginalnym silnikiem bazy danych opartych na wektorach. Podobnie jak *Qdrant*, implementuje algorytm *HNSW*, jednak nie oferuje filtrowania przy jego użyciu, stosując do tego inny algorytm. Nie posiada również wbudowanego pulpitu, a domyślnym sposobem przechowywania danych jest lokalny magazyn plików. Mimo że *ChromaDB* jest natywnym rozwiązaniem dla biblioteki *LangChain*, w zakresie szybkości wyszukiwania nie dorównuje *Qdrant*, który umożliwia znalezienie interesujących rekordów w czasie poniżej milisekundy. Chociaż *ChromaDB* obsługuje metadane, nie wykorzystuje algorytmu *HNSW* do ich filtrowania, co sprawia, że proces ten jest wolniejszy i nieefektywny w przypadku bardziej skomplikowanych filtrów. Dodatkowo warto zaznaczyć, że *Qdrant* oferuje łatwe skalowanie w chmurze lub w klastrze, podczas gdy *ChromaDB* w trybie lokalnym może napotkać ograniczenia.

4.3.5. *Pgvector*

Pgvector nie jest natywną wektorową bazą danych. Jest to rozszerzenie do relacyjnej bazy danych *PostgreSQL*. W porównaniu z innymi bazami, jako rozszerzenie nie istnieje tutaj tylko dostęp do danych, gdzie musi być zdefiniowany wektor. W tej bazie można posiadać tabele związane z przechowanymi dokumentami oraz tabele relacyjne, przykładowo zawierające użytkowników lub wyniki poszczególnych badań. Jest to ogromna zaleta w porównaniu z *Qdrant* oraz *ChromaDB*, jednakże liczne opinie użytkowników tej bazy jasno dają do zrozumienia o jej wolnym trybie działania dla wielu dokumentów. Dla bazy danych może to stanowić dobry kompromis między prędkością a centralizacją danych, jednak jeśli priorytetem jest uzyskanie szybkiej odpowiedzi, nie powinna być ona rozważana w sytuacji, gdy liczba potencjalnych użytkowników będzie bardzo duża.

Oczywiście, tak samo jak *Qdrant*, baza danych implementuje algorytm *HNSW* do przeszukiwania wektorów, jak i wszystkie metryki takie jak, *odległość kosinusowa, iloczyn skalarny, odległość euklidesowa*

4.4. Schemat pełnego modelu RAG

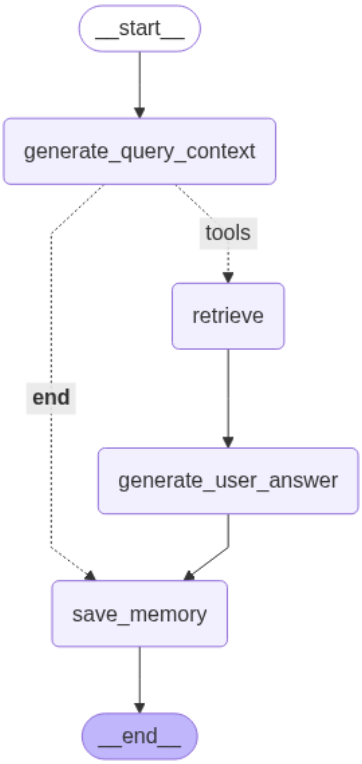
4.4.1. Wybór modelu językowego oraz bibliotek

Najważniejszym elementem RAG jest część generująca odpowiedź. W tym celu używany jest właśnie duży model językowy. Jest on odpowiedzialny za wydzielenie zwróconych wcze-

śniej chunków. Kluczowym jest, aby model rozumiał język polski i właśnie w nim udzielał odpowiedzi. Do tych celów zdecydowano się na wybór modeli od:

- *OpenAi*- płatny model **gpt-4o**, charakteryzował się dobrym zrozumieniem poleceń napisanych w promptach oraz dobrym wyborem kluczowych informacji.
- *Mistral* - bezpłatny model **open-mistral-nemo**, jest to bardzo lekki model przez co jest najszybszy.
- *Meta* - bezpłatny model **llama-3.1-70b-instruct**. Podczas tworzenia prototypu zdarzyło mu się mylić język polski z hinduskim.

Do implementacji systemu badawczego wykorzystano biblioteki *LangChain* oraz *LangGraph* [16]. Są one dostosowane do środowiska *Python* i umożliwiają tworzenie agentów *AI*, takich jak RAG, w oparciu o grafy, w których poszczególne wierzchołki realizują różne zadania (schemat poniżej rysunek 4.9).



Rysunek 4.9.: Pełny schemat stworzonego systemu RAG

Każdy z wierzchołków reprezentuje odrębny krok wykonywany przez agenta.

Kod źródłowy 4.1: Stan aplikacji, w każdym z kroków

```
class RagState(TypedDict):
    """
    Basic State of Rag application cosists
    of array of recent messages and stored memory
    """
    messages: Annotated[Sequence[BaseMessage], add_messages]
    short_conversation_context: str
    parent_ids: Optional[List[str]]
    child_ids: Optional[List[str]]
    user_query: Optional[str]
    retrival_tool: Optional[str]
    mode: Optional[str]
```

W 4.1 została zamieszczona klasa stanu. Jest to klasa, na której bazuje cały graf. W każdym z kolejnych kroków może ulec zmianie poprzez dodanie wiadomości w sekcji *messages* lub uzupełnieniu danych, z których pozyskano kontekst, *child_ids* czy *parent_ids*. Na podstawie tej klasy system może poprawnie przekazać kolejne wiadomości następnym krokom. Poniżej przedstawiono opis poszczególnych kroków widocznych na schemacie 4.9.

4.4.2. Wierzchołek *generate_query_context*

Krok, w którym model językowy decyduje, co zrobić z podanym zapytaniem. Za pomocą wbudowanej w bibliotece opcji, system sprawdza, czy udzielić odpowiedzi natychmiast, czy jednak potrzebne jest pobranie danych z bazy wektorowej. Podczas wybierania modelu językowego istotne było nie tylko to, jak konstruuje odpowiedzi, ale także czy ma możliwość dodania narzędzi. W samym programie wygląda to następująco:

Kod źródłowy 4.2: Przykładowa funkcja narzędzi

```
@tool
def retrieve_data_from_db(user_query: str) -> dict:
    """Na podstawie zapytania danego uzytkownika pobieramy odpowiednie
   fragmenty dokumentow z bazy danych"""

    dalsza czesc funkcji ...
```

Jak możemy zobaczyć z 4.2 każda funkcja narzędzi posiada dwa szczególne elementy:

- Dekorator mówiący bibliotece aby daną funkcję brał pod uwagę jako narzędzia dla modelu.
- Opis danej funkcji. Wykorzystywany on jest przez model językowy do podjęcia decyzji, czy zapytanie użytkownika kwalifikuje się do jej użycia. Całość wyboru podejmuje sam model dlatego istotne jest aby opis danego narzędzia był zwięzły i szczegółowy.

Funkcje narzędzi są szczególnie przydatne w systemie RAG, ponieważ umożliwiają ocenę, jak model radzi sobie z danymi niezwiązanymi bezpośrednio z dokumentem źródłowym. Dzięki temu można zadawać pytania i otrzymywać odpowiedzi w sytuacjach, w których system w pierwotnym rozwiązaniu przeszukiwałby bazę danych i starał się udzielić odpowiedzi na jej podstawie.

Oprócz narzędzi najważniejszym przedmiotem tego kroku jest polecenie przekazywane do modelu:

Główny prompt modelu

Jesteś asystentem inżyniera wodnego z pamięcią długotrwałą.
Twoją najważniejszą funkcją jest odpowiadanie na wiadomości związane z zagadnieniami inżynierii wodnej, takich jak prawo wodne.
Aktualnie w Twojej bazie danych znajdują się dokumenty, takie jak prawo wodne.
Dodatkowo posiadasz opcję przypomnienia sobie wiadomości (recall_memory), które wysłał użytkownik, jak i Ty. Używaj narzędzia (tools) do przypomnienia (recall_memory) – jesteś w stanie przypomnieć sobie najbardziej prawdopodobne wiadomości, do których użytkownik w swoim zapytaniu się odnosi.
Używaj narzędzia (tools) do pobrania z bazy danych (retrieve_data_from_db) – jesteś w stanie pobrać informacje o zagadnieniach z inżynierii wodnej/prawa wodnego z Twojej wektorowej bazy danych.
Wskazówki do używania Twojej pamięci:
1. Aktywnie używaj narzędzi pamięci (recall_memory), aby zbudować kompleksowe zrozumienie użytkownika
2. Rozpoznawaj i uznawaj zmiany w sytuacji lub perspektywie użytkownika na przestrzeni czasu
Wskazówki do pobierania danych z bazy danych związanej z prawem wodnym:
1. Na każde pytanie dotyczące artykułów i ustaw staraj się odpowiadać z bazy
2. Korzystaj z pobierania danych tylko i wyłącznie, jeśli kontekst zapytania użytkownika nawiązuje do prawa lub inżynierii wodnej
Rozmawiaj z użytkownikiem jak ekspert z ekspertem. Zamiast tego, płynnie wkomponuj swoje zrozumienie użytkownika w odpowiedzi. Zwracaj uwagę na subtelne wskazówki i ukryte emocje. Dostosuj styl komunikacji do preferencji użytkownika i jego aktualnego stanu emocjonalnego. Używaj narzędzi (tools), aby zapisywać informacje, które chcesz zachować na kolejną rozmowę. Jeśli wywołujesz narzędzie, cały tekst przed jego użyciem traktowany jest jako wiadomość wewnętrzna. Odpowiadaj dopiero po zakończeniu działania narzędzia i uzyskaniu potwierdzenia, że zakończyło się ono pomyślnie. Nie informuj użytkownika o swoich możliwościach zapisu danych w pamięci.

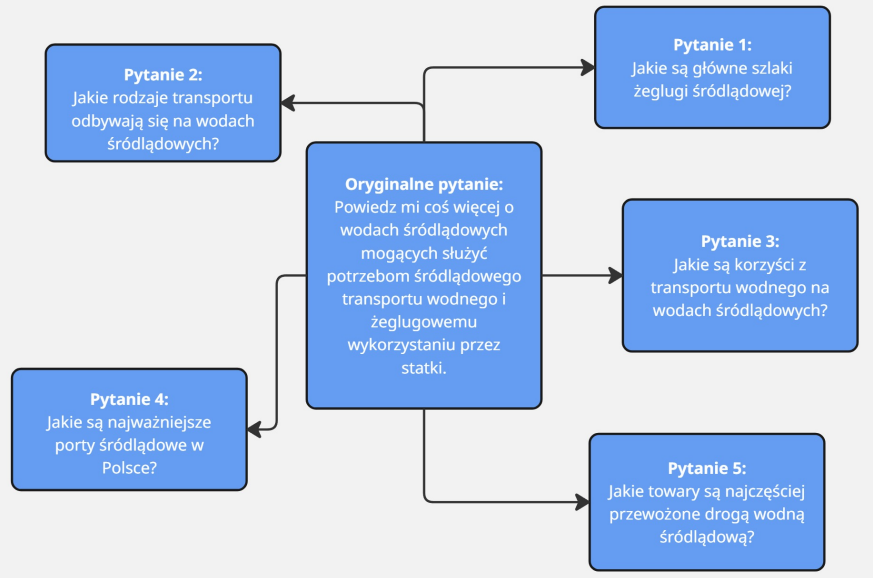
Powyższy prompt posiada kilka głównych elementów, takich jak wstęp tłumaczący mo-

delowi jego główny cel oraz jakie posiada narzędzia do dyspozycji, umożliwiające mu jego realizację. W poleceniu znajdują się wskazówki, w których wyjaśniono dodatkowe szczegóły odnośnie wcześniej omawianych narzędzi, jak i zakończenie, gdzie modelowi zalecono specyficzny styl komunikacji i sposób wypowiedzi. Warto tutaj wspomnieć o długości tego promptа. Z doświadczeń związanych z badaniem zagadnienia asystenta można stwierdzić, iż im dłuższy tekst, tym model łatwiej się gubi, halucynując odpowiedź, zamiast bazować na dokumentach.

4.4.3. Wierzchołek *retrieve*

Retrieve odpowiada za pobranie odpowiednich dokumentów potrzebnych do kontekstu, aby precyzyjnie odpowiedzieć na pytanie użytkownika. Aby zintensyfikować szanse na uzyskanie prawidłowych fragmentów, wykorzystuje się tutaj różne metody doskonalące:

- **Wielokrotność pytań zwane później multi query** - metoda polega na przepisaniu pytania danego użytkownika na inny sposób. Jedno pytanie może zostać wykorzystane jako punkt wyjścia do wygenerowania wielu pytań powiązanych tematycznie, lecz niekoniecznie identycznych pod względem treści lub struktury. Aby wygenerować nowe pytania użyty został ten sam model językowy.



Rysunek 4.10.: Przykład stworzonych pytań za pomocą multi query

Jak widać z rysunku nr 4.10 model nie bazuje na samych słowach z oryginalnego pytania, ale również rozszerza o dodatkowe zwroty. Przykładowo w wyżej pokazanym

pytaniu nie występuję żaden konkretny temat jedynie ogólnik dotyczący transportu na wodach śródlądowych. Model fantazjuje na bazie pytania wymyślając jakie dodatkowe zagadnienia mogą być z tym związane. Dzięki zastosowaniu tej metody na podstawie jednej sentencji od użytkownika zwiększane są szanse na uchwycenie tego, co użytkownik miał na myśli. Mimo iż pomnażamy ilość dokumentów pobranych z bazy danych (każde pytanie pobiera wcześniej ustalone k tekstów) użycie niżej wspomnianego re-rankingu pozwala nam stworzyć odpowiednią kolejność ważności dokumentów. Z pewnością to, że otrzymano kilka odmiennych pytań, nie oznacza, że nie zwracane są identyczne segmenty. Przed ich przekazaniem dalej, konieczne jest usunięcie wszystkich duplikatów.

W celu pozyskania dodatkowych pytań przygotowano dedykowane polecenie skierowane do modelu językowego:

Prompt do modelu

Jesteś asystentem AI. Twoim zadaniem jest wygenerowanie (number_of_queries) różnych wersji podanego pytania w celu późniejszego pobrania danych z bazy danych (zachowaj oryginalne pytanie).
Poprzez wygenerowanie (number_of_queries) różnych wersji jego pytania, masz pomóc użytkownikowi w przezwyciężeniu różnych trudności w wyszukaniu danych, jakie mogą wynikać z jego pytania.
Podaj tylko i wyłącznie nowe pytania, odseparowane poprzez nowe linie, w taki sposób:
Oto (number_of_queries) nowe pytania:
Pytanie1: treść pytania
Pytanie2: treść pytania
Pytanie3: treść pytania
itp.
Oryginalne pytanie: (user_query)

Jak widać z powyżej przedstawionego prompta posiada on kilka zmiennych używanych w systemie odnoszących się do ilości wygenerowanych pytań oraz do oryginalnego pytania. Warto tutaj wspomnieć o wykorzystanej metodzie Few-Shot Prompting w której podano modelowi w jaki sposób powinien zwrócić on listę pytań. Dzięki temu samo wyselekcjonowanie pytań z tekstu jest proste i nie wymaga zaawansowanych funkcji. Poniżej została przedstawiona przykładowa odpowiedź modelu.

Odpowiedź modelu

Oto 5 nowe pytania
Pytanie1: Jakie są główne szlaki żeglugi śródlądowej?
Pytanie2: Jakie rodzaje transportu wodnego są dostępne na wodach śródlądowych?
Pytanie3: Jakie są zalety korzystania z transportu wodnego na wodach śródlądowych?
Pytanie4: Jakie są najważniejsze porty śródlądowe dla transportu wodnego?
Pytanie5: Jakie są wyzwania związane z żeglugą na wodach śródlądowych?

Jak można zauważyć z odpowiedzi udzielonej przez LLM, pytania zostały odpowiednio podzielone dzięki prefiksowi *"Pytanie(nr):"*, co, jak wcześniej zostało wspomniane, ułatwia proces wyekstrahowania pytania do dalszego wykorzystania. Warto jednak wspomnieć o braku oryginalnego pytania, jakim było: *"Powiedz mi coś więcej o wodach śródlądowych mogących służyć potrzebom śródlądowego transportu wodnego i żeglugowemu wykorzystaniu przez statki"* zamieszczonego wyżej na rysunku 4.10. Dzieje się tak z powodu, że funkcja *retrieve_data_from_db* 4.2 jest wcześniej wspomnianym narzędziem. Narzędzie to nie otrzymuje pełnego zapytania użytkownika, ponieważ tekst ten jest wcześniej przetwarzany przez model. Z oryginalnego tekstu model wyselekcjonował tylko najważniejsze fragmenty, jakim było: **wody śródlądowe transport wodny żegluga**. Następnie, na podstawie tej części tekstu, zostały wygenerowane nowe pytania.

- **Re-ranking z użyciem cross-encodera** - system pomaga dodatkowo uporządkować pobrane dokumenty. Mimo, iż silnik bazy zwraca dokumenty na podstawie metryki i porządkuje je od najbardziej do najmniej podobnego do pytania, to często pożądana treść nie jest pierwszym wyborem. W tym celu przeprocesowuje się wszystkie dokumenty przez model typu cross-encoder. Taki model oblicza semantyczne podobieństwo między pytaniem a dokumentami, a następnie tworzy ranking dokumentów, w którym kolejność ustalana jest na podstawie poziomu tego podobieństwa. Dzięki temu do generatora zwracane są odpowiednie dokumenty. Jako model re-rankingu został użyty *radlab/polish-cross-encoder* [17]. Jest to model trenowany na polskich tekstach ze zbioru *polish-sts-dataset* [18]. Jest to szczególnie ważne dla systemu posługującego się tym językiem, ponieważ taki model będzie bardziej precyzyjny w semantycznym prawdopodobieństwie niż model osadzenia tekstów trenowany na wielu językach. Obecnie na rynku nie ma wiele dostępnych takich modeli, w ramach testów zostały sprawdzone inne modele takie jakie:
  - *Alibaba-NLP/gte-multilingual-reranker-base* [19] - z powodu wielojęzyczności wyniki re-rankingu nie odbiegały od kolejności zwracanych z bazy danych, z tego też powodu zaniechano dalszych testów modelu.

  - *mixedbread-ai/mxbai-rerank-base-v2* [20] - model jest również wielojęzyczny . Mimo faktycznej zmiany kolejności dokumentów wysyłanych do tworzenia odpowiedzi w dużej części przypadków główny dokument (zazwyczaj pierwszy) zostawał zawsze odrzucany.

  Podsumowując temat modelów do re-rankingów, zdecydowano się na *radlab/polish-cross-encoder*. Często poprawiał on kolejność zwracanych dokumentów oraz potrafił dobrze dopasować dokumenty do konkretnych słów. Zbiór Prawa Wodnego posiada wysublimowany słownik zatem jeśli użytkownik użył tego samego słowa co znajdowało się w artykule model ustawiał taki dokument znacznie wyżej, czego nie koniecznie dokonywała baza danych.

  Przy użyciu re-rankingu z wyżej wymienionym multi query, model porównuje każdy zwrócony dokument do swojego własnego pytania.

- **Dodatkowy system zwracania pełnego tekstu artykułu** - w wielu przypadkach użytkownik może potrzebować zwróconego kompletnego artykułu. Dostosowanie się do tekstu źródłowego to jedna z funkcji poprawnie działającego systemu RAG. Tak jak wcześniej wspomniano przy omawianiu dostępnych silników bazodanowych ten wykorzystany przez system oferował możliwość dodawania metadanych na zasadzie prostych słowników. Dlatego też każdy fragment tekstów został okraszony dodatkowym tagiem, który mówił z jakiego artykułu pochodzi. Dodatkowo sam model został wzbogacony o dodatkowe narzędzie umożliwiające zwrócenie wszystkich dokumentów związanych z konkretnym artykułem odpowiednio filtrując bazę wektorową.

  Dzięki takiemu rozwiązaniu zapewniane jest dostarczenie pełnej treści artykułu, niwelując przy tym szansę na trafienie w błędny fragment co może się zdarzyć, w wersji bazowania na porównywaniu osadzeń tekstów. Ważne tutaj jest również zapisanie kolejności poszczególnych fragmentów. W następstwie tego można złożyć odpowiedź w kolejności wprost z tekstu źródłowego, co zapewnia iż model językowy nie pominie żadnych fragmentów. Podczas manualnych testów bez zachowywania kolejności, model brał pod uwagę cały tekst który występował po numerze wspomnianego artykułu. Zatem jeśli treść artykułu znajdowała się w trzech fragmentach a fragment rozpoczynający artykuł był drugi to część pierwsza zostawała całkowicie pominięta.

4.4.4. Wierzchołek *generate_user_answer*

Generate_user_answer odpowiada za złożenie semantycznie poprawnej odpowiedzi z dostarczonych dokumentów. Posiada on dodatkowy przejściowy prompt informujący model o sposobie złożenia informacji zwrotnej.

Prompt składania odpowiedzi standardowej

Jesteś botem odpowiadającym na zapytania użytkownika na podstawie dokumentu prawa wodnego. Udało Ci się uzyskać następujące dokumenty z bazy danych:
Dokument z bazy:
(context)
Z ich pomocą odpowiedz na następujące pytanie (question)
Pamiętaj, że masz teraz odpowiedzieć na pytanie, nie używaj żadnych narzędzi (tools), ponieważ wcześniej już ich użyłeś.

W porównaniu z wcześniej wymienionymi poleceniami używanymi w rozwiązaniu ten prompt nie jest najbardziej rozbudowany. Jego głównym zadaniem jest poinformowanie modelu, że jest na kroku udzielenia odpowiedzi, oraz aby nie wywołał przez przypadek jeszcze raz jakiegokolwiek narzędzia. Warto zwrócić uwagę na to, iż nie korzysta się tutaj z pytań stworzonych przez multi query, podawane jest oryginalne pytanie, jakie padło od użytkownika, ponieważ to właśnie na nie system udziela odpowiedzi.

W przypadku zagadnienia odnajdowania całego artykułu, stosowany jest odrębny prompt.

Prompt składania odpowiedzi dotyczącej artykułu

Jesteś botem odpowiadającym na zapytania użytkownika na podstawie dokumentu prawa wodnego. Użytkownik oczekuje, że przytoczysz mu informacje o danym artykule, pamiętaj o tym, aby przy przytaczaniu artykułu przytoczyć wszystkie pobrane informacje.
Artykuły mogą mieć informacje w taki sposób Art. 192 (tekst artykułu) Art. 192a (dalsza część artykułu) Art. 192.1 (dalsza część artykułu) masz uwzględnić wszystkie takie części. Udało Ci się uzyskać następujące dokumenty z bazy danych na temat artykułu:
Document z bazy:
(context)

Z ich pomocą odpowiedz na następujące pytanie (question)
Pamiętaj, że masz teraz odpowiedzieć na pytanie, nie używaj żadnych narzędzi (tools), ponieważ wcześniej już ich użyłeś

Jak możemy zobaczyć z polecenia, model jest instruowany w identyczny sposób, jednakże zostało dodane dodatkowe polecenie tłumaczące, w jaki sposób mogą wyglądać artykuły. Jest to kluczowy element polecenia, ponieważ model nie wie, kiedy kończy się artykuł o danym numerze. W rezultacie braku tej części prompta dodatkowe podpunkty oznaczone tym samym numerem co artykuł traktowałby jako nowe, niezwiązane z pytaniem.

4.4.5. Wierzchołek *save_answer*

Dodatkowy krok w schemacie grafowym do zapisywania bieżących wiadomości do bazy pomocniczej. Nie wpływa on na wynik badań, jedynie jest to funkcja zwiększająca komfort użytkownika, który testował rozwiązanie.

Opisując krótko działanie tego kroku, można je streścić do trzech podpunktów:

- Pobranie tekstu użytkownika i odpowiedzi z systemu RAG
- Zamiana obu tekstów jednocześnie na wektory
- Dodanie ich do osobnej kolekcji w bazie danych

4.4.6. Połączenie kroków w graf

Po przygotowaniu każdej z akcji, jakie są wykonywane w każdym z wierzchołków, w systemie został utworzony poniższy graf.

Kod źródłowy 4.3: Stan aplikacji, w każdym z kroków

```
graph = StateGraph(aqua.RagState)

graph.add_node(aqua.generate_query_context)
graph.add_node("retrieve", ToolNode(aqua.retrive_tools))
graph.add_node(aqua.generate_user_answer)
graph.add_node(aqua.save_memory)

graph.add_edge(START, "generate_query_context")
graph.add_conditional_edges(
    "generate_query_context",
    aqua.route_tools,
    {
        "tools": "retrieve",
        END: "save_memory",
    },
)
graph.add_edge("retrieve", "generate_user_answer")
graph.add_edge("generate_user_answer", "save_memory")
graph.add_edge("save_memory", END)

agent = graph.compile()
```

Na rysunku 4.3 przedstawiono metodę konstrukcji danego grafu. Zdefiniowany wcześniej stan 4.1 jest odpowiednio dodawany, aby system wiedział, jakie zmienne posiada w każdym z kroków. Następnie do grafu dodawane są wierzchołki, takie jak *generate_query_context* czy *generate_user_answer* itp. które zostały omówione w poprzednich podrozdziałach. Warto

zwrócić uwagę na sposób dodawania wierzchołka *retrieve*. Nie jest on standardowy, ponieważ ten wierzchołek wywołuje narzędzia dostępne dla systemu. Aby uzupełnić graf po dodaniu wierzchołków, konieczne jest wprowadzenie krawędzi. Włączenie krawędzi wiąże się z łączeniem istniejących wierzchołków z dodatkowymi, które wskazują początek i koniec grafu, czyli START i END. Definiuje się też krawędź z warunkiem dla wierzchołka z narzędziami. Taka krawędź wymaga dodatkowej funkcji, jaką jest *route_tools*.

Kod źródłowy 4.4: Funkcja route_tools

```
1     def route_tools(state: RagState) -> RagState:
2     """
3     Determine whether to use tools or end the conversation based on the
      last message.
4     """
5     msg = state["messages"][-1]
6     if msg.tool_calls:
7         return "tools"
8
9     return END
```

Funkcja w 4.4 jest bardzo prosta. Na podstawie poprzedniego wywołania sprawdzane jest, czy system nie wywołał żadnego narzędzia (linia w kodzie *if msg.tool_calls:*). Jeśli tak się stało, został dostarczony dodatkowy kontekst i należy przejść do kroku tworzenia odpowiedzi. Na samym końcu kodu 4.3 graf jest kompilowany i tworzony jest agent AquaRAG.

4.5. Podsumowanie

W powyższym rozdziale przedstawiono architekturę zaimplementowanego systemu. Zostały przedstawione moduły, jakie zostały dodane do systemu w celu poprawienia jakości udzielanych odpowiedzi, takie jak re-ranking czy multi query. Opierając się na przykładach, zostały one dokładnie wyjaśnione, aby zapewnić właściwe zrozumienie ich pełnego funkcjonowania. Dodatkowo rozdział został wzbogacony kodem i promptami. Każda z wymienionych pozycji jest szczegółowo omówiona pod kątem funkcjonalności.

Zrozumienie pełnej konstrukcji systemu wraz z funkcjonowaniem poszczególnych modułów jest niezwykle istotne dla prawidłowej interpretacji wyników oraz pojęcia eksperymentów. W kolejnej części przedstawiono eksperymenty przeprowadzone na tej architekturze.

5. Eksperymenty, założenia i ich cele

W poniższym rozdziale zaprezentowane zostały założenia oraz cele prowadzonych eksperymentów. Przedstawiono zbiór ewaluacyjny oraz zdefiniowano konfiguracje systemu, na których prowadzono badania. Przedstawiono także studium przypadku, w którym krok po kroku pokazano działanie całego systemu.

5.1. Założenia

W eksperymentach wprowadzono następujące założenia:

- Stałość zbioru dokumentów źródłowych - doświadczenia były prowadzone na dokumencie Prawa Wodnego. Baza dokumentów nie uległa zmianie podczas prowadzenia badań, dzięki temu przeprowadzono analizę poszczególnych metod indeksowania, wpływu multi query czy re-rankingu.

- Ręcznie przygotowane pytania - zbiór pytań został przygotowany ręcznie. Każde pytanie bazowało swoją odpowiedź na fragmencie tekstu Prawa wodnego. Odpowiedzi były zwięzłe i szczegółowe odpowiednio wyczerpując temat. Ostatecznie przygotowano 30 pytań. Układ pytania i odpowiedzi zaprezentowano w 5.1

Kod źródłowy 5.1: Przykładowe pytanie ze zbioru

```
1     {
2       "id": 19,
3       "question": "Kto ponosi koszty wykonania oraz kto odpowiada za
    utrzymanie dróg, urządzeń do przepędzania zwierząt i przejazdów
    przez wały przeciwpowodziowe?",
4       "answer": "Koszty wykonania ponosi inwestor budujący wał,
    natomiast utrzymanie drogi, urządzeń lub przejazdu spoczywa na
    podmiocie, na którym ciąży obowiązek ich utrzymania."
5     },
6
```

- Zdefiniowane metryki - konfiguracje systemu badane były na podstawie wcześniej zaprezentowanych metryk z rozdziału Metodologia badań. Dzięki zastosowaniu tych samych miar, można było w stanie ocenić kluczowe elementy polepszające jakość udzielanej

odpowiedzi. W przypadku metryk bazujących na modelu oceniający do wszystkich testów zastosowano ten sam model *gpt-4-mini* (oprócz testu gdzie badano wpływ modelu oceniającego).

- Kontrolowanie parametrów systemów - konfiguracja modelu jest modyfikowana w sposób kontrolowany, aby ocenić wpływ poszczególnych modułów na wyniki eksperymentu.

5.2. Cele

W ramach przeprowadzenia eksperymentów na systemie RAG, należy wcześniej zdefiniować dostępne konfiguracje:

- *AquaRAG* - pełny system RAG. Zawiera w sobie indeksowanie z rodzicem, multi query oraz re-ranking. Dodatkowo posiada także narzędzie pozwalające mu znaleźć podany artykuł w bazie i odpowiednio zwrócić jego treść.
- *AquaRAG-multi-query* - system *AquaRAG* bez dodatku re-rankingu. W tej konfiguracji badano wpływ multi query na uzyskane wyniki precyzji pobranych dokumentów.
- *AquaRAG-re-ranking* - system *AquaRAG* bez dodatku multi query, W tej wersji badano wpływ przetwarzania dokumentów przez cross-encoder na wynik uzyskanej odpowiedzi.
- *AquaRAG-no-upgrades* - system *AquaRAG*, gdzie nie zastosowano ani re-rankingu ani multi query. Użyto indeksowania z rodzicem.

Dodatkowo, oprócz powyższego podziału, dodano również wersję systemu *AquaRag-basic*. Jest to wersja zwykłego *AquaRAG*, gdzie zamiast indeksowania z rodzicem, użyto podstawowej wersji.

Celem eksperymentów jest weryfikacja systemu RAG i sprawdzenie udzielanych przez niego odpowiedzi. Zbadanie wpływu poszczególnych modułów na jakość odpowiedzi, wraz z precyzją pobranych dokumentów. Wyodrębnienie modułów, które mają na to największy wpływ. Dodatkowo, porównanie różnych modeli LLM w ramach wcześniej wprowadzonych metryk.

5.3. Przebieg eksperymentów

Przebieg eksperymentów był następujący:

1. Wybranie konfiguracji na jakiej będą prowadzone badania z: *AquaRAG, AquaRAG-multi-query, AquaRAG-re-ranking, AquaRAG-no-upgrades.*
2. Wybranie metody indeksowania z: *Indeksowania z rodzicem, indeksowania podstawowego.*
3. Wybranie modelu językowego dla konfiguracji RAG z *gpt-4o,llama-3.1-70b-instruct, open-mistral-nemo.*
4. Wybranie modelu językowego który będzie ewaluował odpowiedzi z *gpt-4-mini, llama3-8b-instruct, devstral-small-2505.*
5. Wczytanie zbioru z pytaniami.
6. Odpowiedzenie na pytanie i zapisanie informacji w plikach.

Kod źródłowy 5.2: Przykładowa odpowiedź systemu


```
{
    "question": "Kto ponosi koszty wykonania oraz kto odpowiada za utrzymanie dróg, urządzeń do przepędzania zwierząt i przejazdów przez wały przeciwpowodziowe?",
    "system_answer": "Na podstawie dostępnych dokumentów, nie ma bezpośredniej informacji dotyczącej tego, kto ponosi koszty wykonania oraz kto odpowiada za utrzymanie dróg, urządzeń do przepędzania zwierząt i przejazdów przez wały przeciwpowodziowe. Dokumenty zawierają informacje o zarządzaniu wałami przeciwpowodziowymi, w tym o wprowadzaniu zakazów poruszania się po wałach oraz o usuwaniu drzew i krzewów dla zapewnienia ich szczelności i stabilności, ale nie odnoszą się bezpośrednio do kwestii kosztów i odpowiedzialności za utrzymanie dróg i urządzeń związanych z wałami przeciwpowodziowymi.",
    "parent_ids": [
        "8788949c-3f31-4228-b17a-694d53985b6c",
        "bfd8eb46-b8a9-4033-a981-9da5944fd48b",
        "c2e18c87-5a09-4f20-a306-22865f21d117"
    ],
    "child_ids": [
        "0be936cf-4dda-43be-822a-18f21b849126",
        "98607208-acaa-4305-acf9-f224da4dbefe",
        "1aca7d3d-f32d-49ba-bcf9-5b14648b6e11",
        "64a959b0-dbf9-4a5a-a756-44b9060a1a76",
        "abfeab4e-2d25-4add-8212-1fbbd34f1973",
```

```
            "08f54fd7-9608-4992-86e4-2b6026fa07ab",
            "b18d7859-207d-4e96-ab1a-408ea72ce01f",
            "0b9cc3b3-025c-4266-bf88-e57228ca433c",
            "b83ae657-4772-4dd8-8986-433f4af087bf",
            "11a68f27-4a46-4a8f-b3e0-b2060689f8af"
        ],
        "id": 19,
        "reference": "Koszty wykonania ponosi inwestor budujący wał, natomiast utrzymanie drogi, urządzeń lub przejazdu spoczywa na podmiocie, na którym ciąży obowiązek ich utrzymania."
    }
```


7. przeprowadzenie badań dla metryk bazujących na LLM, podstawowych języka naturalnego oraz metryk korzystających z modelu embeddingowego.

5.4. Studium przypadku

Aby lepiej zilustrować działanie proponowanego systemu, poniżej przedstawiono jego funkcjonowanie krok po kroku.

5.4.1. Krok 1: Otrzymanie pytania od użytkownika

Pierwszym krokiem jest pobranie odpowiedzi od użytkownika. Tutaj system decyduje, czy zagadnienie wymaga dodatkowych informacji z wektorowej bazy danych. System dokonuje tego poprzez głównego prompta. W momencie podjęcia decyzji o użyciu wektorowej bazy danych, system izoluje najważniejszą część pytania.

```
Ty: Kto jest właścicielem wód w Polsce i jakie wody uznaje się za publiczne?
INFO:httpx:HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
Update from node generate_query_context
================================== Ai Message ==================================
Tool Calls:
  retrive_data_from_db (call_iWW45XDxNKS0gOttxyVGsCeO)
 Call ID: call_iWW45XDxNKS0gOttxyVGsCeO
  Args:
    user_query: właściciel wód w Polsce, wody publiczne
```

Rysunek 5.1.: Najważniejsza część myśli z pytania użytkownika

Jak można zaobserwować z rysunku nr 5.1 system zdecydował, że najważniejszym zagadnieniem jest właściciel wód oraz wody publiczne. Warto tutaj zauważyć, iż system najpierw

wywołał funkcję narzędzi a potem wyodrębnił do niej kluczowy fragment, który przekazał w argumencie *user_query*.

5.4.2. Krok 2: Multi query

Argument *user_query*, pobrany z pytania przedstawionego na rysunku 5.1, jest następnie przekazywany do modułu multi query. Moduł odpowiednio wywołuje polecenie do swojego modelu językowego i generuje dodatkowe pytania.

Prompt multi query

Jesteś asystentem AI. Twoim zadaniem jest wygenerowanie 5 różnych wersji podanego pytania w celu późniejszego pobrania danych z bazy danych (zachowaj oryginalne pytanie).
Poprzez wygenerowanie 5 różnych wersji jego pytania, masz pomóc użytkownikowi na przezwyciężenie różnych trudności w wyszukaniu danych, jakie mogą wynikać z jego pytania.
Podaj tylko i wyłącznie nowe pytania odseparowane poprzez nowe linie, podaj to w taki sposób "Oto 5 nowych pytań:"
"Pytanie1: treść pytania"
"Pytanie2: treść pytania"
"Pytanie3: treść pytania"
"itp."
"Oryginalne pytanie: Kto jest właścicielem wód w Polsce i jakie wody uznaje się za publiczne?",

Dla przykładowego pytania wygenerowano następujące nowe pytania:

Odpowiedź multi query

Oto 5 nowych pytań:
Pytanie1: Jakie są nazwy właścicieli wód publicznych w Polsce?
Pytanie2: Kto jest właścicielem wód publicznych w Polsce?
Pytanie3: Jakie są firmy lub instytucje, które zarządzają wodami publicznymi w Polsce?
Pytanie4: Kto jest odpowiedzialny za zarządzanie wodami publicznymi w Polsce?
Pytanie5: Jakie są organizacje, które zarządzają wodami publicznymi w Polsce?

Jak można zobaczyć, w nowym zbiorze pytań nie ma oryginalnego pytania zgodnie z wcześniejszym poleceniem.

5.4.3. Krok 3: Pobranie danych z bazy wektorowej

Z każdego pytania eliminowane są niepotrzebne informacje, takie jak nadmiarowe spacje z przodu lub z tyłu. Oraz usuwany jest tekst "Pytanie(nr):", który został wygenerowany zgodnie z poleceniem przekazanym do LLMa. Każde tak zmodyfikowane zapytanie z multi query jest przesyłane do embeddingu. Po wykonaniu osadzenia każde pytanie przeprowadza wyszukiwanie podobnych dokumentów w wektorowej bazie danych. Dla każdego zapytania zwracane jest pięć dokumentów, które są najbardziej zbliżone pod względem wektorów. Kolejnym krokiem jest tworzenie par pytanie–dokument, co pozwala odpowiednio przygotować zbiór danych do etapu re-rankingu.

5.4.4. Krok 4: Re-ranking

Po pobraniu wszystkich dodatkowych kontekstów, pracę zaczyna moduł re-rankingu. W pętli wyliczane są podobieństwa pytania oraz zwróconego dokumentu. Model zwraca listę posortowanych tekstów, gdzie pierwszy posiada najwyższy wynik. W przypadku pracy z indeksowaniem z rodzicem należy teraz jeszcze raz zweryfikować, czy nie ma duplikatu danego fragmentu. Po sprawdzeniu, z lokalnej bazy rodziców pobierane są teksty, aby stworzyć odpowiedź.

5.4.5. Krok 5: Zwrócenie odpowiedzi użytkownikowi

Kiedy przygotowany jest już kontekst odpowiedzi, trafia on do ostatniego Prompta:

Prompt generujący odpowiedź

Jesteś botem odpowiadającym na zapytania użytkownika na podstawie dokumentu prawa wodnego. Udało Ci się uzyskać następujące dokumenty z bazy danych:
Dokumenty z bazy: (tutaj jest kontekst)
Z ich pomocą odpowiedz na następujące pytanie: **Kto jest właścicielem wód w Polsce i jakie wody uznaje się za publiczne?**
Pamiętaj, że masz teraz odpowiedzieć na pytanie, nie używaj żadnych narzędzi (tools), ponieważ wcześniej już ich użyłeś

Przed zwróceniem odpowiedzi system zapisuje wynik w sposób zaprezentowany w 5.2. Następnie, po wykonaniu polecenia, system zwraca następującą odpowiedź:
Właścicielem wód w Polsce mogą być Skarb Państwa, inne osoby prawne lub osoby fizyczne. Wody morza terytorialnego, morskie wody wewnętrzne, śródlądowe wody płynące oraz wody podziemne stanowią własność Skarbu Państwa. Wody, które są własnością Skarbu Państwa lub jednostek samorządu terytorialnego, są uznawane za wody publiczne. Śródlądowe wody

płynące będące wodami publicznymi nie podlegają obrotowi cywilnoprawnemu, z wyjątkiem przypadków określonych w ustawie

Dana odpowiedź była ewaluowana przez inżyniera wodnego, który stwierdził jej zasadność.

5.5. Podsumowanie

W powyższym rozdziale zaprezentowane zostały założenia oraz cele eksperymentów. Zdefiniowano konfiguracje systemu, które różnią się modułami czy metodami indeksowania. Przedstawiono przebieg eksperymentów, gdzie po kolei wytłumaczono, w jaki sposób zostały przeprowadzone. Ten rozdział został wzbogacony przez prezentację techniki, w jakiej zostały zapisane pytania w systemie. Dodatkowo przeprowadzono szczegółową analizę przypadku, opisując krok po kroku, jakie procesy zachodzą w systemie.

6. Ewaluacja

W poniższym rozdziale zaprezentowano wyniki eksperymentów przeprowadzonych na stworzonych konfiguracjach. Każdy wynik został opatrzony interpretacją, wskazującą najważniejsze obserwacje. Dodatkowo w zaprezentowanych tabelach wytłuszczono najlepsze wyniki dla poszczególnych metryk.

6.1. Pytania badawcze

Ewaluacja systemu miała odpowiedzieć na następujące pytania badawcze:

- Jak dobrze można ocenić system RAG bez czynnika ludzkiego?
- Wpływ modułów systemu na uzyskane odpowiedzi?
- Jaka konfiguracja systemu zapewnia jak najmniejsze halucynowanie?
- Wpływ modelu LLM na wyniki?
- Wpływ sposobu indeksowanie na wyniki?

6.2. Wyniki ewaluacji z wykorzystaniem metryk

Do każdej z wymienionych tu metryk przekazano minimalnie dwa z czterech wymienionych parametrów:

- Zwracane fragmenty tekstów przez RAG.
- Odpowiedź udzielona przez system.
- Pytanie użytkownika.
- Odpowiedź referencyjna.

6.2.1. Precyzja

Przy badaniu precyzji wybrano dwie metryki bazujące na modelu językowym. Precyzja bez referencji sprawdza, jak precyzyjna jest odpowiedź w porównaniu z zwróconymi fragmentami, natomiast precyzja z referencją weryfikuje odpowiedzi systemu z odpowiedzią ręczną.

System	Precyzja z referencją	Precyzja bez referencji
AquaRAG	**0.9472**	**0.9056**
AquaRAG-multi-query	0.8167	0.8833
AquaRAG-re-ranking	0.9306	0.8861
AquaRAG-no-upgrades	0.8750	0.7528

Tabela 6.1.: Porównanie systemów RAG dla **precyzji**.

Jak można zobaczyć z tabeli 6.1 najlepszy wynik osiągnięty został dla systemu z wszystkimi modułami. Kluczowy jest re-ranking. Systemy, które go posiadają, znajdują się na miejscu pierwszym i drugim pod względem precyzyjności. Zwiększa on wynik dla obu konfiguracji, gdzie był zastosowany. Konfiguracja korzystająca z modułu multi query zwróciła najgorszy i bardzo odbiegający od innych wynik precyzji z referencją. Jest to spowodowane działaniem tego modułu, czyli rozszerzaniem pytania o dodatkowe tematy. Nie jest tutaj używany re-ranking; korzystamy tylko z miary silnika bazy, dlatego system daje gorszy wynik, bo do modelu generatywnego przekazujemy dokumenty niekoniecznie najbliższe pytaniu użytkownika.

6.2.2. Recall

Badanie *recall* mówi, jak wiele zwróconych fragmentów tekstów jest poprawnych oraz jakich ewentualnie brakuje. Przeprowadzono badanie standardowego *recall*, gdzie porównywano twierdzenia znalezione w referencji i pobranych fragmentach. Dodatkowo zbadano *recall* encji, jednakże jego wynik jest zaskakująco niski.

System	Recall	Recall encji
AquaRAG	0.7500	0.1400
AquaRAG-multi-query	0.6461	0.1612
AquaRAG-re-ranking	**0.8722**	**0.2353**
AquaRAG-no-upgrades	0.6422	0.0704

Tabela 6.2.: Porównanie systemów RAG dla ***recall***.

Czytając tabelę 6.2 można stwierdzić, iż oceniający model nie był w stanie wyznaczyć konkretnych encji z badanego tekstu Prawa Wodnego. Można zatem określić, iż taka metryka ma

sens jedynie dla tekstów, gdzie można jasno wyróżnić różne encje. Wracając do wyników, najlepszy rezultat osiągnął system z modułem re-rankingu; co ciekawe, znacząco odskoczył on od wyniku systemu z wszystkimi modułami, gdzie również się znajduje. *AquaRAG-multi-query* oraz system *AquaRAG-no-upgrades* dały porównywalne rezultaty. Można zatem wywnioskować, dlaczego pełny system osiągnął gorszy wynik od systemu re-rankingowego. Moduł multi query osłabił działanie pełnej konfiguracji, osłabiając część wyszukiwania tego RAGa. Dzieje się tak, ponieważ ten moduł generuje różne rodzaje tego samego pytania, przez co pobierane fragmenty mogą odbiegać od oryginalnego zapytania, zwracając inne dokumenty niż wymagane, które nie zawsze zostaną odrzucone przez re-ranking.

6.2.3. Wrażliwość na szum

W badaniu wrażliwości na szum istotne jest osiągnięcie możliwie najniższej wartości metryki. Celem jest uzyskanie odpowiedzi zawierających jak najwięcej faktów przy jednoczesnym minimalizowaniu liczby błędnych lub generowanych w sposób fantazyjny informacji. Właśnie tę funkcję spełnia omawiana metryka. Dzięki niej możliwa jest identyfikacja scenariuszy, w których może wystąpić nadmierna generacja nieprawdziwych treści.

System	Wrażliwość na szum
AquaRAG	0.1900
AquaRAG-multi-query	0.2490
AquaRAG-re-ranking	0.2049
AquaRAG-no-upgrades	**0.1524**

Tabela 6.3.: Porównanie systemów RAG dla **wrażliwości na szum**.

Jak można zobaczyć z tabeli 6.3 najlepszy rezultat został osiągnięty dla systemu podstawowego. Ten system zwraca fragmenty tekstu jak najbliższe zadanemu pytaniu, przez co system nie dostaje informacji, które są niezwiązane z pytaniem. Nie jest to zaskoczenie, ponieważ badane tutaj jest, jak bardzo nasz system kłamie, a nie czy czegoś brakuje w odpowiedzi. W przypadku multi query dodatkowe pytania faktycznie mogą, a nawet powinny, generować szum. Nie jest to jednak wada tego rozwiązania, bo część domyślania się, co użytkownik miał na myśli, jest równie istotna w systemie RAG.

6.2.4. Podobieństwo semantyczne

Badanie podobieństwa semantyczności opierało się na modelu odpowiedzialnym za embedding. Jako model został wybrany ten sam, co został użyty podczas osadzania tekstu do bazy danych, czyli model od Nvidii. Podczas porównywania wektorów zastosowano *odległość kosinusową*.

System	Podobieństwo semantyczne
AquaRAG	0.8272
AquaRAG-multi-query	0.7961
AquaRAG-re-ranking	**0.8374**
AquaRAG-no-upgrades	0.7764

Tabela 6.4.: Porównanie systemów RAG dla **podobieństwa semantycznego**.

Jak możemy zobaczyć z wyników w 6.4, pełny system i system z re-rankingiem dały porównywalne wyniki. Multi query tylko nieznacznie zwiększył skuteczność w porównaniu z podstawowym. Dzięki modułowi re-rankingu, który również bazuje na porównywaniu tekstów osadzonych, jesteśmy w stanie przybliżyć odpowiedź systemu do tej docelowej. Początkowo można pomyśleć, że ten moduł nie powinien tutaj mieć znaczenia, nic bardziej mylnego. Model był szkolony tylko na polskich tekstach, zatem radzi sobie lepiej w znalezieniu dokumentów potrzebnych do udzielenia odpowiedzi.

6.2.5. Wierność

Badanie wierności opiera się na sprawdzeniu ilości twierdzeń w odpowiedzi, popartych zwróconymi dokumentami, z ilością wszystkich twierdzeń z odpowiedzi.

System	Wierność
AquaRAG	0.9022
AquaRAG-multi-query	0.8514
AquaRAG-re-ranking	**0.9062**
AquaRAG-no-upgrades	0.7676

Tabela 6.5.: Porównanie systemów RAG dla **wierności**.

Warto tutaj zauważyć, iż wynik zaprezentowany w 6.5 dla systemu pełnego oraz re-rankingu jest niemal identyczny, system *AquaRAG-multi-query* jest nieznacznie gorszy od nich, a system podstawowy znacząco odbiega od innych. Można stwierdzić, iż re-ranking jest kluczowy dla tej metryki. Dzięki niemu jesteśmy w stanie zmienić kolejność zwracanych dokumentów, na których model następnie bazuje przy udzielaniu odpowiedzi. W tych systemach więcej informacji jest faktycznie poparte dokumentami.

6.2.6. Trafność odpowiedzi

Trafność odpowiedzi jest badaniem, gdzie sprawdzana jest trafność odpowiedzi w stosunku do danych wprowadzonych przez użytkownika. Jest to metryka bazująca na modelach LLM,

gdzie na podstawie odpowiedzi systemu są generowane dodatkowe pytania, a następnie między nimi a oryginałem liczona jest *odległość kosinusowa*.

System	Trafność odpowiedzi
AquaRAG	0.3248
AquaRAG-multi-query	**0.3310**
AquaRAG-re-ranking	0.3211
AquaRAG-no-upgrades	0.3048

Tabela 6.6.: Porównanie systemów RAG dla **trafności odpowiedzi**.

Jak można zobaczyć z tabeli 6.6 wyniki trzech systemów, *AquaRAG*, re-rankingowego oraz *AquaRAG-multi-query*, są zbliżone. Jest to zaskakujący wynik, ponieważ wydawałoby się, iż system z multi query znacząco odbiegnie od pozostałych z powodu architektury generowania dodatkowych pytań. Dzieje się tak, ponieważ nawet przy generowaniu dodatkowych pytań i pobieraniu odmiennych dokumentów, model wciąż koncentruje się na udzieleniu odpowiedzi na pierwotne pytanie użytkownika.

Ta metryka na przestrzeni całych badań dała najgorsze wyniki. Nie oznacza to jednak, że system działa źle. Odwołując się do poprzednich badań 6.3, możemy wywnioskować, że system mało fantazjuje. Co za tym idzie, udziela odpowiedzi na pytanie bez zbędnych faktów czy zbędnych dywagacji.

6.2.7. Zgodność z faktami

W poniższym badaniu weryfikowano, jak dobrze system radzi sobie z generowaniem odpowiedzi zgodnych z faktami. Metryka zgodności z faktami pozwala porównać wygenerowaną odpowiedź z odpowiedzią referencyjną, określając stopień ich zgodności. W tym celu odpowiedzi są najpierw rozkładane na poszczególne twierdzenia, a następnie przy użyciu wnioskowania językowego (*natural language inference*) określa się, które twierdzenia w odpowiedzi zgadzają się z twierdzeniami w odpowiedzi referencyjnej.

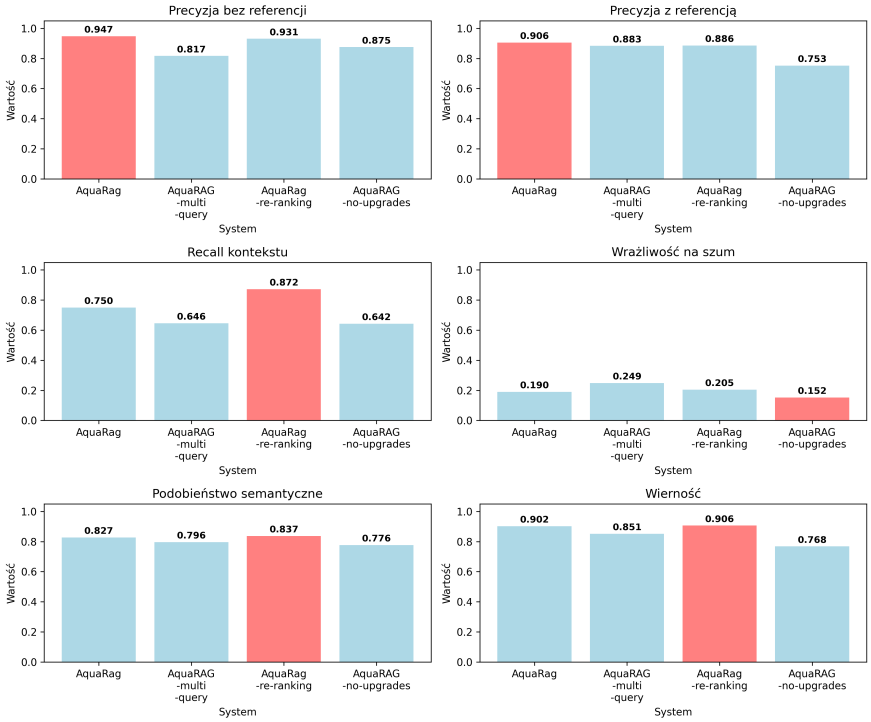
System	Zgodność z faktami
AquaRAG	0.4883
AquaRAG-multi-query	0.4023
AquaRAG-re-ranking	**0.5523**
AquaRAG-no-upgrades	0.2147

Tabela 6.7.: Porównanie systemów RAG dla **zgodności z faktami**.

Jak można zobaczyć z tabeli 6.7 żaden z systemów nie zwrócił zadowalających wyników. Najlepiej poradził sobie system z re-rankingiem, który znacząco odbiegł od pozostałych,

jednak dalej jest to wynik minimalnie powyżej średniej. Dramatycznym wynikiem zaś może pochwalić się podstawowy system. To znaczy, że zwracane w nim dokumenty nie zawierały twierdzeń, które zostały później użyte w odpowiedzi. Wynik tej metryki dla wszystkich systemów jest bardzo słaby i świadczy o tym, iż zwracane dokumenty nie posiadały wszystkich informacji lub podczas tworzenia odpowiedzi model pominął ważne fakty. Sprawdzając również wynik z poprzedniej tabeli 6.5 gdzie badana była wierność, możemy wywnioskować, iż system bazował na dokumentach, ale nie dostarczał wszystkich istotnych faktów. Prawdopodobna zmiana w ilości tekstu zwracanego do prompta lub zmiana samego prompta mogłaby poprawić tę metrykę.

Podsumowując część wyników z metryk bazujących na LLM i modelu embeddingowym, można zaprezentować następujący wykres.



Rysunek 6.1.: Ocena systemu z wykorzystaniem poszczególnych metryk

6.2.8. ROUGE

Badanie za pomocą metryki *ROUGE* nie bazuje na modelu językowym, tylko na n-gramach. Nie jest to też idealna metryka do badania systemu RAG, co można zauważyć poprzez zaprezentowane wyniki. Jednakże użyto jej jako odniesienia, gdzie nie zwracano uwagi na wynik ogólny systemu, a to, jak on różni się od innych konfiguracji.

- *ROUGE Precyzja*

System	ROUGE-1	ROUGE-2	ROUGE-L
AquaRAG	0.4321	0.3274	0.3958
AquaRAG-multi-query	0.3730	0.2580	0.3206
AquaRAG-re-ranking	**0.4762**	**0.3569**	**0.4309**
AquaRAG-no-upgrades	0.2669	0.1468	0.2121

Tabela 6.8.: Porównanie systemów pod względem ***ROUGE precyzja***.

Jak widać z tabeli 6.8 ogólny wynik precyzji jest słaby. Dzieje się tak, ponieważ metryka nie uznaje zdań identycznych semantycznie. Można jednak z niej zauważyć tendencję jaką cechują się wszystkie wyniki a mianowicie najlepszy wyniki posiadają systemy z re-rankingiem. W samej precyzji odbiegają one aż o około 0.1 od wyników multi query. Dodatkowo można też stwierdzić, iż system podstawowy posiadał najgorzej zredagowane odpowiedzi nie używające tych samych słów co z dokumentów.

- *ROUGE Recall*

System	ROUGE-1	ROUGE-2	ROUGE-L
AquaRAG	0.6184	**0.4682**	0.5727
AquaRAG-multi-query	0.5272	0.3656	0.4664
AquaRAG-re-ranking	**0.6375**	0.4649	**0.5792**
AquaRAG-no-upgrades	0.4125	0.2096	0.3297

Tabela 6.9.: Porównanie systemów pod względem ***ROUGE recall***.

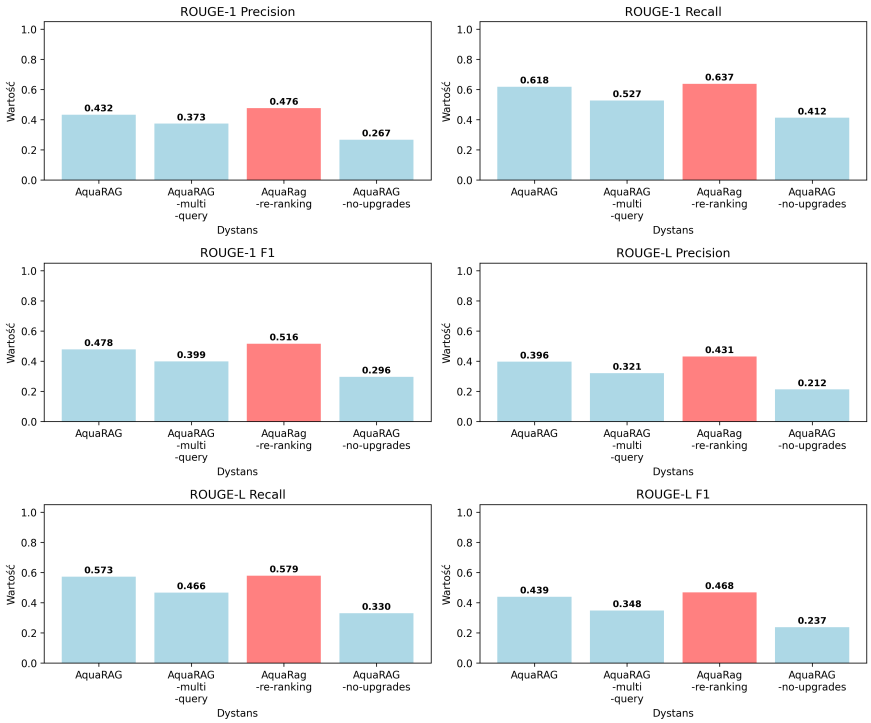
Z wyników *recall* z 6.9, można również stwierdzić, że najważniejszy jest re-ranking. Znowu wyniki z tym modułem znacząco odbiegły od systemów, w których go brakowało.

- *ROUGE F1*

System	ROUGE-1	ROUGE-2	ROUGE-L
AquaRAG	0.4778	0.3614	0.4394
AquaRAG-multi-query	0.3986	0.2778	0.3481
AquaRAG-re-ranking	**0.5157**	**0.3854**	**0.4682**
AquaRAG-no-upgrades	0.2965	0.1587	0.2371

Tabela 6.10.: Porównanie systemów pod względem ***ROUGE F1***.

Ostatnią zaprezentowaną metryką *ROUGE* jest i jej wynik znajduje się w 6.10. Jak można zobaczyć, znowu najbardziej istotny jest moduł re-rankingu.



Rysunek 6.2.: Wykres podsumowujący metrykę *ROUGE*

6.2.9. BLEU

Badania za pomocą metryki *BLEU* okazały się bezcelowe. Niniejsze wyniki są rezultatami z pojedynczych n-gramów i jak możemy zauważyć, wszystkie systemy mają ten sam pomiar. Jest to spowodowane definicją tej metryki, czyli porównywaniem wyrazów. W przypadku, gdy słowo w tekście referencyjnym jest inne, np. synonim, to metryka nie zakwalifikuje tego pozytywnie. Ten sam przypadek dzieje się, kiedy zdania są identyczne semantycznie. W przypadku, kiedy szyk zdania ulega zmianie, znowu *BLEU* stwierdzi niepoprawność. Dodatkowo modele LLM mają tendencję do dodawania dodatkowych epitetów do zdań, co również sprawia problem wg. tej metryki. Podsumowując, badanie *BLEU* jawnie pokazało ograniczenia tej metryki w ocenie jakości generowanego tekstu, zwłaszcza w kontekście naturalnego języka, gdzie możliwe są różne formy wyrażenia tej samej treści. *BLEU* jest wrażliwe na literalne dopasowanie n-gramów, co powoduje, że synonimy, zmiany szyku zdań czy dodatkowe epitetowe wyrażenia generowane przez modele LLM nie są uwzględniane jako poprawne. W efekcie metryka ta może zaniżać ocenę faktycznej jakości semantycznej generowanych odpowiedzi,

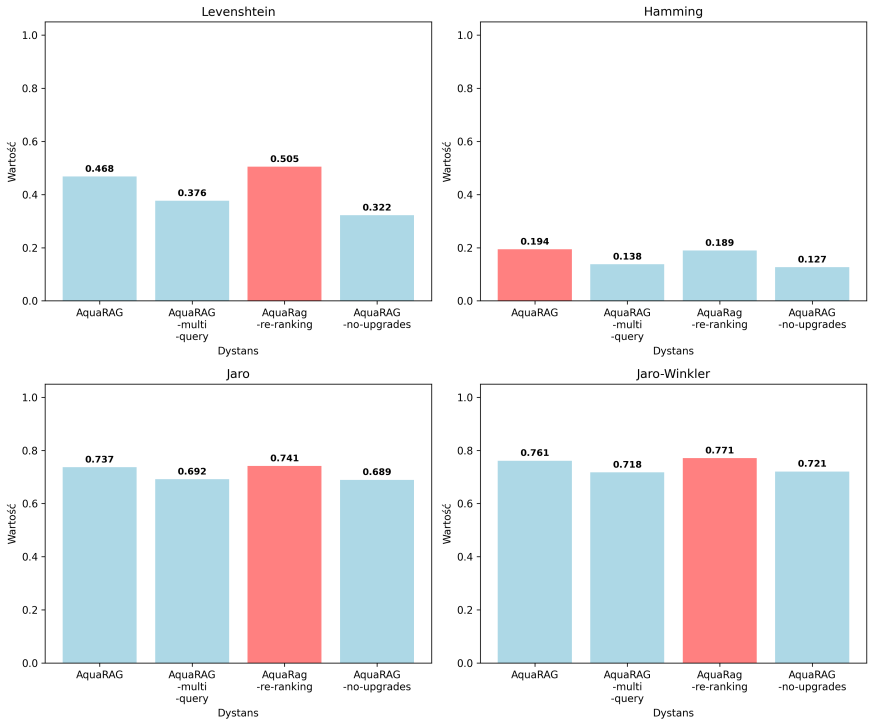
nie odzwierciedlając rzeczywistości.

System	BLEU
AquaRAG	0.0716
AquaRAG-multi-query	0.0716
AquaRAG-re-ranking	0.0716
AquaRAG-no-upgrades	0.0716

Tabela 6.11.: Porównanie systemów pod względem ***BLEU***.

6.2.10. Metryki odległości tekstowej

Podczas eksperymentów zbadano działanie systemu dla standardowych metryk w przetwarzaniu języka naturalnego. Wyniki tych doświadczeń zaprezentowano poniżej:



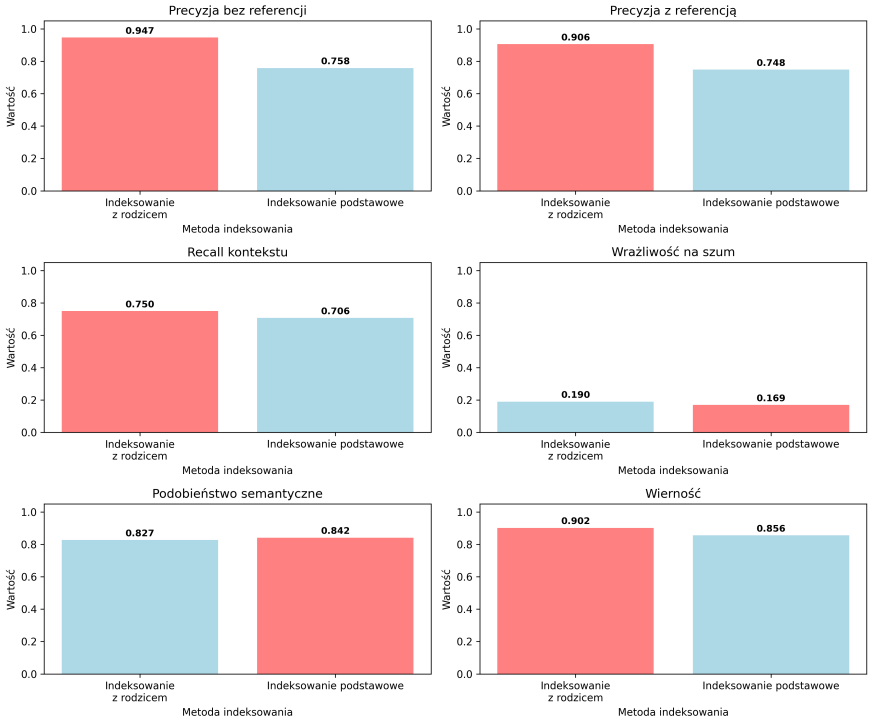
Rysunek 6.3.: Porównanie wyników systemu dla poszczególnych metryk odległości

Jak widać z rysunku 6.3 nie wszystkie metryki nadają się do ewaluacji RAG. Przykładowo, metryka *Hamminga* nie zwróciła żadnego wyniku powyżej 0.2. Jest to zrozumiałe, ponieważ starsze metody ewaluacji generowanego tekstu nie wpasowują się idealnie w najnowsze realia

tej dziedziny. Jednakże metryka *Jaro* i *Jaro-Winklera* zwróciły wynik, który można analizować. Dzieje się tak, ponieważ te metryki są dużo bardziej rozbudowane oraz nie bazują na usuwaniu lub dodawaniu elementów, tylko na ich transpozycji. Jest to zdecydowanie pozycja, z którą warto rozważać w wnioskach.

6.2.11. Porównanie sposobów indeksowania

W ramach badań zbadano również wpływ sposobu indeksowania na zwracane wyniki. Sprawdzono *indeksowanie z rodzicem* oraz *podstawowe*. Poniżej zaprezentowano wyniki metryk:



Rysunek 6.4.: Wyniki systemu dla poszczególnych metod indeksowania

Jak można zaobserwować z wykresu 6.4, *indeksowanie z rodzicem* radzi sobie znacznie lepiej niż podejście *podstawowe*. W najważniejszych metrykach, takich jak precyzja czy *recall*, rozwiązanie to znacząco przewyższa metodę podstawową. Warto jednak zauważyć, że podejście bazowe charakteryzuje się znacznie mniejszą wrażliwością na szum. Dzieje się tak, ponieważ ilość tekstu dostarczanego do promptu jest w tym przypadku istotnie mniejsza niż w *metodzie z rodzicem* (fragmenty różnią się wielkością około trzykrotnie).

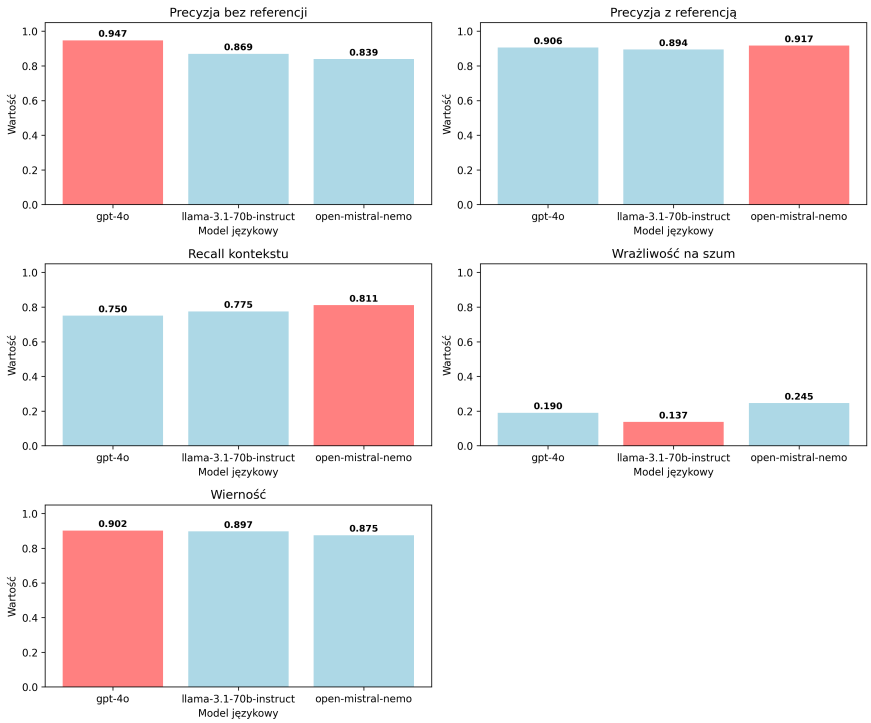
Interesującym wynikiem jest również bardzo niska wartość wskaźnika wierności dla rozwiązania *podstawowego* (0.6756), która nie w pełni koreluje z niską podatnością na szum. Oznacza to, że system generuje stwierdzenia niepoparte zwróconym tekstem, co wskazuje na problem z halucynowaniem mimo ograniczonego kontekstu wejściowego.

6.2.12. Porównanie wyników dla różnych modeli językowych

W ramach pracy przeprowadzono badania wpływu modelu generatywnego na działanie *AquaRAG*. Sprawdzono w tym celu trzy modele, a były to:

- *gpt-4o* od *OpenAi*.
- *llama-3.1-70b-instruct* od *Mety*.
- *open-mistral-nemo* od *MistralAi*.

Na poniższym wykresie przedstawiono wyniki. Ze względu na zastosowanie systemu z multi query, wartości takie jak *recall* i podobieństwo semantyczne różnią się między sobą, gdyż są zależne od dodatkowych pytań wygenerowanych przez model.



Rysunek 6.5.: Wydajność systemu dla różnych modeli językowych

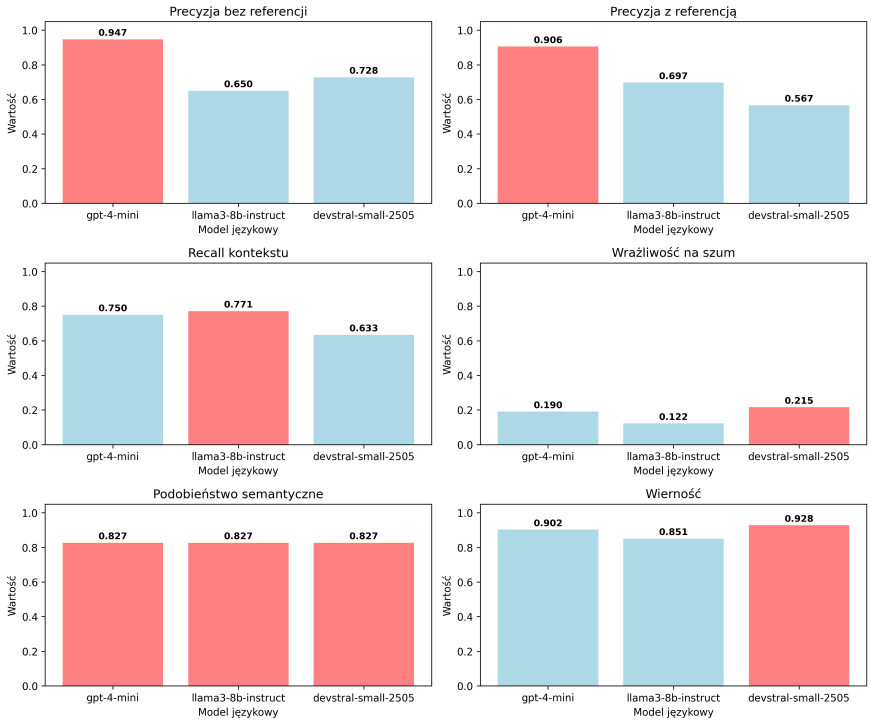
Jak można zauważyć z wykresu 6.5 wybranie modelu do systemu nie jest oczywiste. Weryfikując precyzję, można dostrzec, iż w ogólnym rozrachunku gpt radzi sobie najlepiej. Ciekawy jest spadek precyzji bez referencji dla modeli *llama* oraz *open-mistral-nemo*, podczas gdy precyzja z referencją nie odbiega od wyników modelu *OpenAI*, a w przypadku *Mistrala* nawet je przewyższa. Oznacza to, że modele nie wykorzystały w pełni potencjału z zwróconego tekstu, jednakże nie znaczy, że odpowiedziały błędnie, co potwierdza podobieństwo semantyczne, gdzie wynik jest niemal identyczny dla wszystkich modeli.

Warto też poruszyć wynik *recall*, jaki uzyskał model *open-mistral-nemo*. Mimo najwyższego wyniku (czyli zwrócenia poprawnych dokumentów) model nie uzyskał najwyższego wyniku wcześniej wspomnianej precyzji bez referencji.

Interesujący jest też rezultat wrażliwości na szum. Wszystkie modele uzyskały względnie mały wynik, nie halucynując odpowiedzi. Najniższy wynik jednak uzyskał model *llama*, co jest zaskakujące, ponieważ nie osiągnął najlepszych wyników w żadnej z precyzji.

Odpowiedni wybór modelu do tworzenia odpowiedzi może znacząco zwiększyć uzyskane rezultaty naszego systemu. W zależności od tego, czy priorytetem jest osiągnięcie najwyższej precyzji, czy też minimalizacja halucynacji, należy odpowiednio dobrać model. Jeśli kluczowa jest dokładność odpowiedzi, warto rozważyć wybór modelu o wyższej precyzji, natomiast w sytuacjach, gdzie istotniejsza jest spójność i unikanie błędnych informacji, lepszym rozwiązaniem będzie model charakteryzujący się niższą podatnością na szum.

Podczas badania zweryfikowano również wpływ modelu oceniającego na wyniki poszczególnych metryk. Otrzymany rezultat zaprezentowany jest w tabeli poniżej. Tak samo jak w poprzednim badaniu wybrano modele od *OpenAI - gpt-4-mini, Mety - llama3-8b-instruct czy MistralAI - devstral-small-2505*. Niestety modele *llama* oraz *devstral* nie poradziły sobie z obliczeniami kilku metryk, takich jak *recall* encji, trafność odpowiedzi czy zgodność z faktami (wejście tych modeli było za małe, aby obsłużyć te metryki). W zamieszczonej tabeli nie podano wyniku dla podobieństwa semantycznego, ponieważ jak wcześniej wspomniano, nie jest ono metryką związaną z modelem językowym i dany wynik byłby identyczny dla wszystkich modeli.



Rysunek 6.6.: Porównanie wyników dla rozwiązania systemu AquaRAG dla modeli oceniających

Jak można spostrzec z wykresu 6.6 różne modele zwracają różne wyniki metryk. Najlepsze wyniki zostały zwrócone dla modelu *gpt-4-mini*. Pozostałe modele odbiegają niemal w każdej metryce. Te wyniki dają jasno do zrozumienia, iż model ewaluacyjny jest niemal identycznie ważny jak model użyty do generacji odpowiedzi. Przy wybraniu modelu, nie radzącego sobie z obliczeniami metryk, otrzymywany jest zakłamany wynik. Dodatkowo rezultaty mówią jasno, iż ocenianie systemu poprzez same metryki jest niewystarczające i potrzebny jest czynnik ludzki, czynnik ewaluacji z ekspertem.

6.3. Ewaluacja z ekspertem

Podczas badań przeprowadzono również sesję z ekspertem inżynierii wodnej, znającym bardzo dobrze Prawo Wodne. Ekspert skierował do systemu szereg pytań o zróżnicowanym stopniu trudności i następnie ocenił, na ile uzyskane odpowiedzi spełniają jego oczekiwania. Pytania zostały skategoryzowane następująco:

- *Proste* - Pytania te odnosiły się do jednego artykułu, zatem jednego konkretnego miejsca w dokumencie.

- *Średnio-trudne* - Były to w dużej mierze pytania dotyczące różnych artykułów, a tym samym kilku sekcji dokumentu.
- *Trudne* - Pytania dotyczyły licznych artykułów i potrzebna wiedza była rozmieszczona w całym dokumencie.

Początkowo badano zachowanie systemu dla pytań prostych. Ekspert m.in. zadał następujące pytania:

- Na czym polegają melioracje wodne z artykułu 195?
- Na podstawie, którego artykułu robi się zgłoszenie wodno prawne?
- Co powinien zawierać operat wodno prawny?

Na wszystkie powyższe i inne pytania system odpowiedział dobrze. Specjalista stwierdził, że system poprawnie zinterpretował fragmenty tekstu, gdzie znajduje się kontekst do odpowiedzi. Oraz zignorował te, w których go nie ma. Zaprezentowane zostało również dogłębne działanie modułów. Fachowiec stwierdził, ogromną przydatność multi query. Jego zdaniem to ulepszenie pozwalało systemowi domyślać się zagadnienia pytania, bez dużego opisu, co znacznie poprawiało zwróconą odpowiedź. Mimo wyższego szumu, jaki system z tym modułem posiada, dodatkowe opisy nie wpłynęły negatywnie na opinię eksperta. Problematyczne okazały się pytania trudne, ponieważ system zwraca tylko i wyłącznie konkretną ilość fragmentów (konkretnie 3) i nie udało się uwzględnić całości odpowiadających fragmentów. Brakujące fragmenty były widoczne w rezultacie, gdzie system udzielał wybrakowanej odpowiedzi, niesatysfakcjonującej eksperta. Przykładem tego może być:

- Pytanie: Jakie dokumenty należy dostarczyć aby zalegalizować urządzenie wodne?
- Odpowiedź: Aby zalegalizować urządzenie wodne, należy dostarczyć dokumenty potwierdzające:
  1. Tytuł prawny podmiotu wnoszącego sprzeciw do dysponowania urządzeniem wodnym.
  2. Legalność wykonania urządzenia wodnego.
  3. Legalność korzystania z wód z wykorzystaniem urządzenia wodnego.

  Te dokumenty są załączane do sprzeciwu, który można wnieść do właściwego organu Wód Polskich.

Jak można zobaczyć, z odpowiedzi nie nawiązuje ona wprost do pytania i na nie nie odpowiada. W dokumencie Prawa Wodnego istnieje wiele sekcji, gdzie tematem artykułu są dokumenty. To pytanie posiada rozproszoną odpowiedź po wielu takich artykułach, dlatego też system miał problem z odpowiedzią. Dzieje się tak, ponieważ system RAG nie jest przewidziany na pobieranie dużej ilości dokumentów i zwracania ich do prompta. W przypadku

takiego rodzaju pytań należałoby zmienić architekturę, gdzie zwracane fragmenty dokumentu są krótsze, ale jest ich więcej.

Podsumowując, fachowiec stwierdził przydatność systemu i ocena jego skuteczności była pozytywna. Stwierdził, że system byłby przydatny w jego dziedzinie, gdzie często musi zaglądać do dokumentów Prawa Wodnego. Poinformował, że *AquaRAG* posiada potencjał, jeśli chodzi o to pole.

6.4. Podsumowanie

W powyższym rozdziale zdefiniowano pytania badawcze oraz zaprezentowano wyniki metryk LLM oraz metryk przetwarzania języka dla różnych konfiguracji *AquaRAG*. Zweryfikowano w nich najważniejsze elementy systemu, od wyszukiwania po generację. Zweryfikowano również znaczenie czynnika ludzkiego w ewaluacji. Każde badanie zawierało własne wnioski wynikające z analizy rezultatów, które w kolejnej sekcji tej pracy zostaną zestawione i przejrzyście podsumowane.

7. Podsumowanie

W poniższym rozdziale znajduje się podsumowanie całej pracy. Zostanie skonstruowane ogólne podsumowanie wyników oraz przedstawione dalsze prace rozwojowe systemu. Dodatkowo uwypuklone zostaną osiągnięcia, jakie udało się uzyskać, tworząc ***AquaRAG****.*

7.1. Osiągnięcia

W niniejszej pracy badawczej udało się opracować oraz zrealizować system RAG. Przeanalizowano badania naukowe odnoszące się do niego. Znalezione w badaniach propozycje implementacji, jak i ewaluacji, znalazły swoje zastosowanie w późniejszej pracy. Dodatkowo, poprzez zapoznanie się z początkowymi prototypami systemu, dogłębnie zrozumiano jego działanie.

Zaprezentowano proces konstrukcji systemu oraz opisano poszczególne moduły. Z powodzeniem zaimplementowano poprawnie multi query oraz re-ranking. Dzięki powstałym modułom udało się wprowadzić kilka konfiguracji do eksperymentów w celu znalezienia elementu, który ma największy wpływ na jakość udzielanych odpowiedzi. Poprzez zaimplementowanie różnych sposobów indeksowania zbadano wpływ dostarczonych informacji na budowę odpowiedzi.

Szczegółowo zaprezentowano metodologię prowadzenia badań dla takiego systemu oraz obszernie objaśniono metryki, jakie są używane w eksperymentach. Pomyślnie zbadano system za pomocą miar z oceniającym modelem językowym, jak i z użyciem starszych metod przetwarzania języka, takich jak odległość Jaro czy Jaro-Winklera. Zweryfikowano wpływ różnych modeli LLM na ogólne działanie systemu, gdzie sprawdzono jedne z najpopularniejszych rozwiązań. Sprawdzono również wpływ modeli oceniających na ogólną ewaluację systemu za pomocą metryk z użyciem LLM.

Pomyślnie zbadano prototyp na spotkaniu z ekspertem ds. inżynierii wodnej. Podczas sesji uzyskano cenne uwagi dotyczące możliwego rozwoju *AquaRAG*.

7.2. Wnioski z przeprowadzonych badań

Podsumowując wyniki doświadczeń, można jasno zdefiniować najlepszy system. Jest nim *AquaRAG*, posiadający najlepsze wyniki precyzji odpowiedzi. W prawdzie *AquaRAG-re-*

ranking góruje nad nim w innych metrykach takich jak podobieństwo semantyczne czy wierność, to różnice nie są zbyt duże.

Jak można było zaobserwować z wykresu 6.1 w większości metryk *AquaRAG* otrzymywał najlepsze rezultaty bądź bliskie bycia najlepszymi. W przypadku precyzji jest jednoznacznym liderem, chociaż rozwiązanie *AquaRAG-re-ranking* posiada bardzo zbliżone wyniki. Oznacza to, że systemy skutecznie udzielały odpowiedzi na pytania, precyzyjnie odnotowując, co należy pobrać z kontekstu zwróconych fragmentów.

Z rezultatu tego eksperymentu można stwierdzić, iż re-ranking jest szczególnie ważny do poprawnego działania systemu. Jego użycie znacząco zwiększyło precyzję (o około 0.5 w porównaniu z kolejnym najwyższym wynikiem). A dodatkowa analiza zwróconych chunków pozytywnie wpłynęła na dostarczony kontekst, co widać po badaniu *recallu kontekstu*. Re-ranking wpływa również pozytywnie na *wierność*, z jaką są tworzone odpowiedzi. Metryka również bazuje na badaniu dostarczonych fragmentów tekstu, zatem jest to kolejne potwierdzenie, że zmiana szyku po rankingu stworzonym przez bazę danych jest szczególnie ważna. W przypadku *wrażliwości na szum* najlepiej radził sobie podstawowy system *AquaRAG-no-upgrades*. Różnica między innymi systemami nie jest marginalna, ale weryfikując inne badania, nie widać wskazań, aby system często ulegał halucynacji.

W ramach badań sprawdzano też metryki niebazujące na modelu oceniającym LLM. Nie dawały one wysokich wyników, jeśli chodzi o *precyzję* czy *recall*, jednakże nadal pozwalają na porównanie tych wyników wśród konfiguracji.

Jak można było zobaczyć z rysunku 6.2 wyniki dla tej metryki nie są zadowalające. LLM często odpowiada na pytanie, dodając dodatkowe słowa, nie wpływające na meritum wypowiedzi, dlatego też wyniki tych metryk są tak słabe. Możliwe jest jednak porównanie różnych konfiguracji systemu dla tego badania. Z rysunku 6.2 doskonale widać, że kluczowym modułem jest re-ranking. Znacząco zwiększa on wszystkie wartości tej metryki.

Oprócz *ROUGE* zbadano również podstawowe metryki odległości. Wyniki tych metryk są szczególnie interesujące, ponieważ dla miary *Jaro* oraz *Jaro-Winklera* system zwrócił wyniki w granicach 0.69 i 0.77. Jest to szczególnie dobry wynik, jak na tak proste metryki odległości tekstowej. Dla miar *Hamminga* czy *Levenshteina* wyniki nie były już tak dobre. Zgodnie z przewidywaniami odległość *Hamminga* okazała się niedobrym podejściem. Miara opierająca się na fakcie, że teksty muszą mieć równą długość, nie daje dobrych wyników, gdyż wygenerowane teksty często różniły się długością od odpowiedzi referencyjnej. Odnośnie dystansu *Levenshteina* można powiedzieć, że jest to miara, która dobitnie pokazała, jak ważny jest re-ranking. Dostarczone przez system konteksty do złożenia były tak zbliżone do referencyjnej odpowiedzi, że system nie potrzebował dużej ilości zamian, usunięć czy dodawania liter, aby uzyskać te same zdania. Podsumowując, miary dystansu oraz *ROUGE* nie są najlepszymi metrykami do ewaluowania systemów RAG. Mimo to wyniki tych metryk odzwierciedlają tendencję, jak można również zobaczyć z innych eksperymentów, najważniejszym modułem jest re-ranking.

Dodatkowo zweryfikowano wpływ sposobu indeksowania na precyzję i *recall* pobranych

dokumentów oraz złożenia z nich odpowiedzi. Z wykresu 6.4 można jasno wywnioskować, że najlepszym sposobem indeksowania jest *indeksowanie z rodzicem*. Dzięki dokumentom typ dziecko skutecznie zwiększona została precyzja trafienia interesującego nas zagadnienia. Widać to z wyniku metryki *recall*, gdzie podstawowe rozwiązanie posiada gorszy wynik aż o około 0.05. Dzięki lepszemu wyborowi dokumentów rośnie również *precyzja bez referencji*, gdzie porównywano referencyjną odpowiedź z chunkami zwróconymi przez system. Patrząc na *precyzję z odniesieniem do referencji*, można również zauważyć, że model pobierał więcej danych z fragmentów.

Weryfikując *wrażliwość na szum*, możemy dostrzec przewagę *podstawowego sposobu*, jednakże jest ona minimalna. Nie powodowało to zbytnich problemów dla systemów z *indeksowaniem z rodzicem*, ponieważ *precyzja* pozostawała na wysokim poziomie. Dodatkowo w oczy rzuca się również przewaga *podstawowego rozwiązania* w *podobieństwie semantycznym*. Znowu różnica jest minimalna i nie wpływała na wyniki pozostałych badań. Przyglądając się metryce *wierności*, można zauważyć kolejną zaletę *indeksowania z rodzicem*. Znacząco przewyższył *podstawowy sposób*, bo aż o około 0.05. Sugeruje to, że gorsza metoda posługiwała się wiedzą niepotwierdzoną zwróconymi dokumentami. Przyrównując to do *wrażliwości na szum*, można wywnioskować, że *metoda podstawowa* halucynuje fakty, gdzie *metoda z rodzicem* poruszała zagadnienie niekoniecznie związane z pytaniem, (dlatego wyższy wynik szumu, ale też wyższy wynik wierności). Jest to spowodowane obszerniejszym kontekstem, jakim wtedy system dostaje.

W badaniach również zweryfikowano wpływ LLMa na uzyskane odpowiedzi. Zdecydowano się przebadać trzy różne modele, w tym jeden płatny od *OpenAi*.
Z wykresu 6.4 można z łatwością wyłonić najlepszy LLM a jest nim *gpt-4o*. Posiada on najlepszą *precyzję bez referencji* ze wszystkich modeli, odskakując od nich aż o około 0.1. Pomimo iż nie jest liderem *precyzji z referencją*, to wynik jest na identycznym poziomie dla wszystkich modeli. Jego dobre rezultaty z *precyzji* są szczególnie interesujące, ponieważ w metryce *recall* poradził sobie zdecydowanie najgorzej. Mimo to, to właśnie on był w stanie wywnioskować jak najwięcej z dokumentów podanych mu w promptowym poleceniu.

W przypadku halucynacji również objawia się jako lider. Chociaż różnice w wierności dla wszystkich kandydatów nie są wysokie. W kategorii wrażliwości na szum niestety gpt znacząco odbiegł od najlepszego modelu *llama*. Różnica 0.05 może spowodować, iż zwracana przez niego odpowiedź nie zawiera poprawnych faktów. Jednakże zestawiając to z wiernością, można dojść do wniosku, iż ten LLM był bardziej opisowy i dodawał zbędne informacje.

Z punktu widzenia finansowego darmowe modele nie odbiegają w ogromnym stopniu od płatnej wersji. Mimo iż w ogólnym rozrachunku dały gorszy wynik, nie wyklucza to ich z bycia dobrym budżetowym rozwiązaniem.

Podsumowując, najlepszym systemem okazał się *AquaRAG*, gdzie zastosowano *indeksowanie z rodzicem* wraz z modelem płatnym *gpt-4o*. Kluczowym modułem, jaki pozwolił mu osiągnąć najlepsze wyniki, był moduł re-rankingu. Znacząco zwiększał *precyzję*, *recall* czy *wierność*; w niektórych przypadkach nawet przewyższając *AquaRAG*, który dodatkowo po-

siadał multi query.
Ważnym elementem tego systemu okazał się również sposób zapisywania danych w bazie, czyli *indeksowanie z rodzicem*. W porównaniu z podstawowym rozwiązaniem znacząco zwiększa praktycznie wszystkie metryki (tylko szum był gorszy).
Chociaż nie miało to aż tak dużego znaczenia, wybór modelu językowego był istotny. Płatny model *gpt-4o* poradził sobie najlepiej ze wszystkich badanych egzemplarzy. Z pewnością zwiększył on *precyzję*, z jaką *AquaRAG* udzielał odpowiedzi. Jednakże nie zawsze był najlepszy we wszystkich metrykach. Dla *recallu kontekstu* znacząco odbiegł od obu innych kandydatów. Jest to zdecydowanie miejsce, gdzie należałoby system ulepszyć.

7.3. Przyszłe prace

W dalszych pracach nad projektem można by zastanowić się nad implementacją poniższych ulepszeń:

- Hybryda modeli - dodanie aby inny model tworzył zapytanie multi query niż *gpt-4o*. Bazując na rezultatach kandydatów *open-mistral-nemo* tworzy lepsze pytania, z których później zwrócone dokumenty są bardziej zbieżne z zapytaniem użytkownika

- Poprawa jakości odpowiedzi na pytania *trudne* (patrz Ewaluacja z ekspertem) - należałoby wprowadzić dodatkowy etap, który oceniałby, czy odpowiedź dostarczona przez system przy użyciu mniejszej liczby dłuższych fragmentów jest lepsza niż odpowiedzi złożone z wielu małych fragmentów. Zapobiegłoby to nie udzieleniu dogłębnej odpowiedzi na pytanie użytkownika, gdzie dany kontekst jest rozsiany poprzez cały tekst źródłowy.

- Zaimplementowanie webowego interfejsu - niestety w ramach badań nie udało się zaimplementować webowego interfejsu. Podczas sesji z ekspertem okazało się to problematyczne i ciężko było mu samemu obsługiwać system. W celu prowadzenia dalszych badań z nim należałoby przygotować odpowiedni interfejs gdzie mógłby zadawać pytania

- Większa ilość dokumentów - w niniejszych badaniach sprawdzano system bazujący na dokumencie Prawa Wodnego. Jednakże w dziedzinie inżynierii wodnej znajduje się wiele książek nie związanych z częścią prawną. Można by dodać pozycje związane ściśle z nauką i matematycznymi obliczeniami. System musiałby obsługiwać formuły oraz dodatkowe zdjęcia (schematy), gdzie wytłumaczona zostałaby treść.

- Rozszerzenie metryk - podczas dalszych prac można by rozszerzyć ilość metryk lub nawet stworzyć własne. Samo ręczne metryki mogłyby dotyczyć stricte zagadnienia inżynierii wodnej karząc system za nie zwracanie odpowiedzi w konkretnym żargonie.

- Rozszerzenie systemu o rozumowanie - system mógłby posiadać rozumowanie w celu lepszego udzielenie odpowiedzi. Użytkownik mógłby wtedy dopytać o dodatkowe informacje i bazując na poprzednich konwersacjach system udziela głębszej odpowiedzi.

Słownik terminów

BART BART (Bidirectional and Auto-Regressive Transformers) to model językowy typu Transformer, który łączy podejście autoenkoderowe i autoregresywne, wykorzystywany m.in. do tłumaczenia, streszczania i generowania tekstu . 7

chunk Fragment dokumentu źródłowego używany do tworzenia odpowiedzi. Chunki znajdują się w wektorowej bazie danych pod postacią wektorów. 9, 14, 21, 22, 24, 31, 66, 67

cross-encoder Cross-encoder to architektura modelu (najczęściej opartego na Transformerach), stosowana głównie w zadaniach re-rankingu dokumentów oraz oceny podobieństwa semantycznego . 36, 42

embedding Proces zamiany tekstu w wektor, poprzez wykorzystanie odpowiedniego modelu. 13, 19, 20, 22, 24, 27, 44, 46, 51, 54

Few-Shot Prompting Technika gdzie LLM otrzymuje kilka przykładów zadania w promptcie, aby poprawnie wygenerować odpowiedź na nowe, podobne pytanie . 35

fine-tuning to proces douczania LLMa do nowego, specyficznego zadania lub zbioru danych, przy użyciu mniejszej ilości danych niż przy pierwotnym treningu . 5

generator Część systemu RAG, gdzie generowana jest odpowiedź . 6, 25, 36

halucynowanie Halucynowanie modelu to zjawisko, tworzenia niepoprawnych odpowiedzi twierdząc ich poprawność.. 2, 15, 34, 49, 59, 60, 66, 67

istotne dokumenty Dokumenty, których kontekst jest potrzebny do udzielenia odpowiedzi. 14

LLM Duży model językowy nazwa pochodzi od skrótu angielskiego large language model. 1, 3, 5, 7, 8, 11, 13, 14, 16, 19–21, 36, 42, 44, 46, 49, 52, 54, 56, 63, 65–67

multi query Metoda polegająca na rozbiciu wejściowego pytania na kilka zbliżonych i na ich podstawie wyszukania dokumentów z bazy wektorowej. 11, 12, 37, 38, 40–42, 45, 46, 51–53, 55, 59, 62, 65, 68

n-gram Ciąg kolejnych n elementów (np. słów, znaków lub tokenów) w danym tekście. Przykładowo, dla zdania "Ala ma kota" 2-gramy to: "Ala ma", "ma kota" . 18, 19, 54, 56

narzędzie Dodatkowa funkcja wykorzystywana przez model w celu udzielenia odpowiedzi. 8, 12, 32–34, 36, 38, 40, 45

Nvidia Firma z USA produkująca karty graficzne i udostępniające modele języków lub Embedding-owe. 51

open source Oprogramowanie, którego kod źródłowy jest publicznie dostępny i może być dowolnie używany, modyfikowany i rozpowszechniany. 1

overlapping Overlapping (nakładanie się fragmentów) polega na dzieleniu tekstu w taki sposób aby początek jednego fragmentu był końcem poprzedniego a koniec jednego fragmentu początkiem . xiii, 21, 22

prompt Polecenie przekazywane do modelu językowego . 7, 31, 33–35, 37, 38, 40, 44, 46, 54, 58, 62, 67

RAG Skrót od Retrieval-Augmented Generation (Generacja Wspomagana Wyszukiwaniem). xiii, xv, 2, 3, 5–9, 11–15, 17–21, 26, 30, 31, 33, 37, 39, 42, 43, 49–54, 57, 62, 65, 66

seq2seq Seq2seq (sequence-to-sequence) to architektura sieci neuronowych, która przekształca jedną sekwencję danych wejściowych w inną sekwencję danych wyjściowych, np, w tłumaczeniu maszynowym . 5–7

T5 Założeniem T5 (Text-to-Text Transfer Transformer) jest konwersja wszystkich zadań przetwarzana języka do formatu tekst-tekst, w którym zarówno dane wejściowe, jak i wyjściowe są ciągami tekstowymi . 15

twierdzenie Pojęcie z biblioteki Ragas. Twierdzeniem jest część zdania w której stwierdzany jest fakt. 14–16, 52–54

Bibliografia

[1] Krunal Vaghasiya. „The Latest ChatGPT Statistics and User Trends (2022-2025)". W: *Wisernotify* (2025).

[2] Aleksandra Piktus Patrick Lewis Ethan Perez. „Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". W: *Acm Digital Library* (2020).

[3] Zhicheng Dou Yujia Zhou Zheng Liu. „Boosting the Potential of Large Language Models with an Intelligent Information Assistant". W: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. 2024.

[4] Kai Zhang i inni Hao Yu Aoran Gan. *Evaluation of Retrieval-Augmented Generation: A Survey*. Springer, Singapore, 2025.

[5] Chowdary Nattem Buchepalli Praneeth Mohana Eshitha. „Optimization of Customer Feedback Summarization Using Large Language Models (LLM) and Advanced Retrieval-Augmented Generation". W: *IEEE Explore* (2025).

[6] https://docs.ragas.io/en/v0.3.2/. Wer. 0.3.2.

[7] https://huggingface.co/vectara/hallucination_evaluation_model.

[8] maxbachmann. *RapidFuzz*. https://pypi.org/project/RapidFuzz/.

[9] Harrison Chase. *Langchain*. https://python.langchain.com/docs/introduction/. Wer. 0.3.25. 2025.

[10] https://chunkviz.up.railway.app/.

[11] Nvidia. *nv-embed-v1*. https://build.nvidia.com/nvidia/nv-embed-v1.

[12] Andrey Vasnetsov André Zayarni. *Qdrant vector database*. https://qdrant.tech/. Wer. 1.14.0. 2025.

[13] Chroma. *Chroma*. https://www.trychroma.com/.

[14] *pgvector*. https://github.com/pgvector/pgvector.

[15] „Hierarchical Navigable Small Worlds (HNSW)". W: *Pinecone* (2025).

[16] Harrison Chase. *Langgraph*. https://langchain-ai.github.io/langgraph/concepts/why-langgraph/. Wer. 0.6.0. 2025.

[17] *radlab/polish-cross-encoder*. https://huggingface.co/radlab/polish-cross-encoder.

[18] *radlab/polish-sts-dataset*. https://huggingface.co/datasets/radlab/polish-sts-dataset.

[19] *Alibaba-NLP/gte-multilingual-reranker-base*. https://huggingface.co/Alibaba-NLP/gte-multilingual-reranker-base.

[20] *mixedbread-ai/mxbai-rerank-base-v2*. https://huggingface.co/mixedbread-ai/mxbai-rerank-base-v2.